

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. GeoJSON.org. "GeoJSON." <https://geojson.org/>.

- **Required pages:** Whole webpage
- **Estimated reading time:** 5 minutes

2. Temple University. "What are Shapefiles?" 20 Sept. 2022. <https://guides.temple.edu/c.php?g=895881>.

- **Required pages:** Whole webpage
- **Estimated reading time:** 10 minutes

Recommended Readings

1. IBM. "What is Geospatial Data?" <https://www.ibm.com/topics/geospatial-data>

- **Recommended pages:** Whole webpage
- **Estimated reading time:** 15 minutes

Lesson 1 - Geospatial Data

December 25, 2025

MODULE 6 | LESSON 1

1 Geospatial Data

Reading Time	60 minutes
Prior Knowledge	Basic Python
Keywords	Geospatial data, raster data, vector data, GIS, CRS

1.1 1. Introduction

In this lesson, we are going to introduce geospatial data. This type of data is tagged with location information in the form of a coordinate system. We will go through the basic concepts of geospatial data. Then, we will demonstrate how to use Python to process geospatial data for further analysis. The knowledge we learn from this lesson will also help us study for this module's upcoming lessons on satellite imagery and climate data. # **2. Geospatial Data**

1.2 2.1. What is Geospatial Data?

Geospatial Data is a type of dataset containing information about features on Earth's surface. What is a feature? Things or events we can observe on Earth's surface are called features. For example, a tree, a building, a road, and a river are examples of features. These examples are static features that do not move or change very quickly. Features can also be dynamic, like a car's travel route or the path of a hurricane. There are two components of feature information in a geospatial dataset: **feature position** and **feature attributes**.

Feature position tells us where this feature is located on a coordinate system on Earth's surface. Depending on data types within the geospatial dataset, there are different methods of presenting feature position information. We'll discuss this more later on in this lesson. **Feature Attributes** tells us other non-geolocation information, such as a building's height, year built etc. Another example can be a hurricane's speed and pressure at certain time.

When discussing feature position, we talked about representing a location on a coordinate system for Earth surface. In the next section, we are going to learn more about the coordinate systems used in geospatial data. ## **2.2. Geospatial Data in Financial Analysis** In recent years, applying geospatial data in financial analysis has become more and more popular. Geospatial data can provide different aspects of information about financial activities that traditional financial data cannot provide. Geospatial data links economic outcomes, the movement of people or objects, and

natural environment change together. Here are some examples of how to apply geospatial data in financial analysis

Insight into the Movement of People and Objects Geospatial data offers greater insight into the movement of both people and objects. For example, by understanding the frequency and timing of customer foot traffic for a retail store, financial analysts can predict sales and make informed decisions about the company's performance.

Evaluation of Credit Risk Geospatial data can be used by financial institutions to evaluate the risk of underwriting a commercial property loan. It can analyze the repayment histories of adjacent commercial properties.

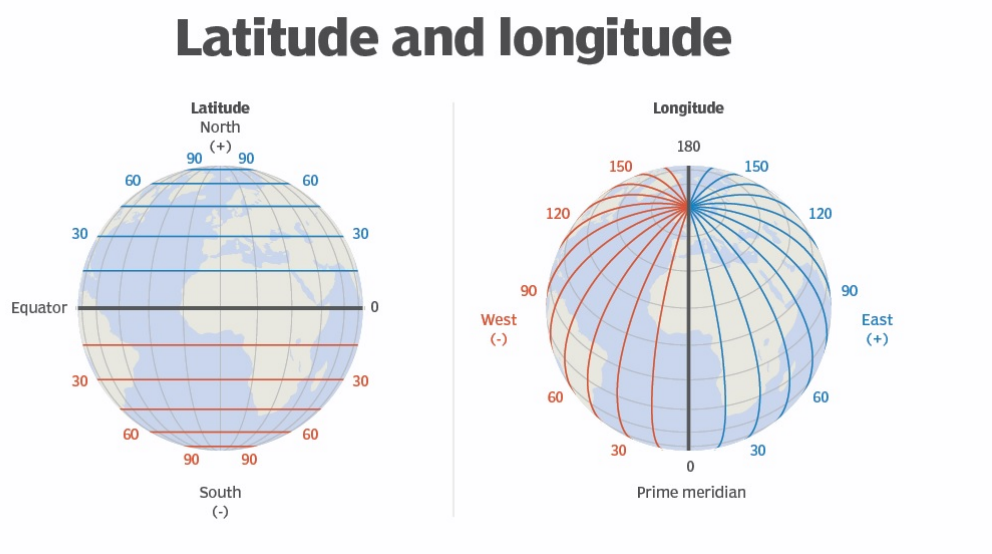
Selection of Bank Branch Location By analyzing the movement and shopping routes of a community's residents, a bank can identify the optimal location for a bank branch that will cross most residents' travel routes.

3. Coordinate Reference Systems A **coordinate reference system (CRS)** is a coordinate system to describe location information on Earth's surface (Awati, 2022). There are various CRS system frames you can use, but the most popular approach is using latitude and longitude to form a grid system on Earth's surface.

Latitude lines are horizontal lines across the Earth to show how far a location is away from the equator with respect to the directions of north and south. The 0 degree line of latitude is located at the equator. It divides Earth into the Northern Hemisphere above the equator and the Southern Hemisphere below the equator.

Longitude lines are vertical lines across the Earth to show how far a location is away from the prime meridian with respect to directions of west and east. The 0 degree line of longitude is located at the prime meridian. It divides the Earth into the Eastern Hemisphere and the Western Hemisphere. Figure 1 demonstrates this coordinate system.

Figure 1. Earth's Latitude and Longitude Coordinate System

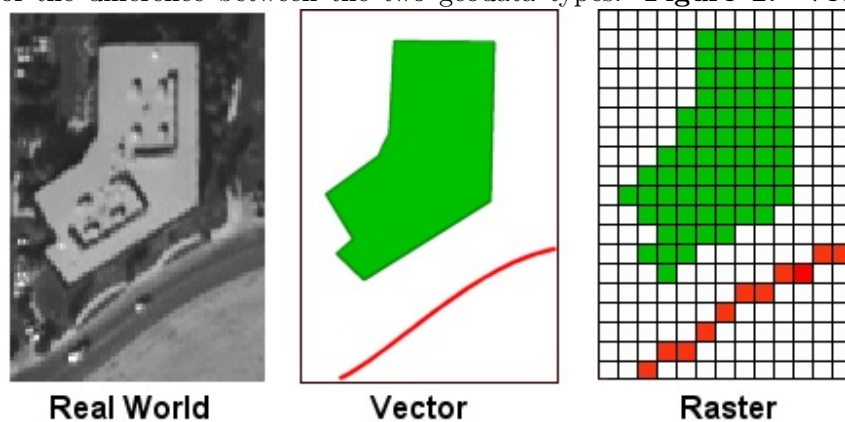


Source: TechTarget When referencing a location on a CRS, the general consensus is to start first with latitude and then longitude in the form of (latitude, longitude). There are two ways to present latitude-longitude information. The first method is degrees, minutes, and seconds (DMS). For example, the coordinates using DMS for New York are (40° 43' 50.1960'' N, 73° 56' 6.8712'' W). The ° denotes degree. The ' denotes minute, and the '' denotes second. The second method is decimal degrees (DD). The coordinates using DD for New York are (40.730610, -73.935242). There are two main differences between DMS and DD. The first difference is that DMS and DD use different symbols to represent the direction information of a location. DMS uses characters to describe which hemisphere the location is in (N vs S; W vs E). DD uses + and - signs to describe

the hemisphere location. In the DMS method, New York's direction coordinates are (N, W). In the DD method, New York's direction coordinates are (+,-). We can use Figure 1 to understand how the DD method assigns +/- signs. On the left part of the figure, we can see that if a location is in the Northern Hemisphere, the latitude sign is +. If the location is in the south, the latitude sign is -. The concept is the same when applied to longitude. The second difference is that DMS is in degrees and DD is in decimals. We can easily convert between the two methods. To convert DMS to DD, first we keep the degree number. Then, we divide the minute number by 60 and divide the second number by 3600. We then add the results of the two divisions and add them back to the degree number. The final number will be the number in DD. Sometimes, which hemisphere the data is in is indicated by the variable name or header, and the values of data points are all positive in the table. In this case, we need to manually adjust the values of the data points to properly represent the hemisphere location in the DD method. In Python, we will use the DD method to present coordinate information. You can easily find the DD coordinates for a place in Google Maps. # 4. **Types of Geospatial Data** In the last section, we introduced the latitude-longitude CRS. In the feature position section, we talked about how different geodata types will use different methods to present location information. There are two main geodata types: raster data and vector data.

Raster data divides a feature into a grid system. Each grid unit is called a pixel or a cell. Raster data stores feature attributes based on each pixel. In the satellite imagery lesson, we will talk more about raster data.

Vector data stores feature attributes based on feature shape. A feature can have a point shape, a line shape, or a polygon shape. In this lesson, we'll focus on vector data. Before moving on to the next topic of vector data, let's take a look at Figure 2 to get a visual understanding of the difference between the two geodata types. **Figure 2. Vector Data vs**



Raster Data

Source:

CUNY Hunter # 5. **Vector Data** In the previous section, we showed how we can present a location using a CRS. So far, we've only talked about a point location, which means a dot on the latitude-longitude coordinate system. Apart from a point shape, we can also include line shapes and polygon shapes with a CRS. Figure 3 demonstrates examples of a point shape, a line shape, and a polygon shape.

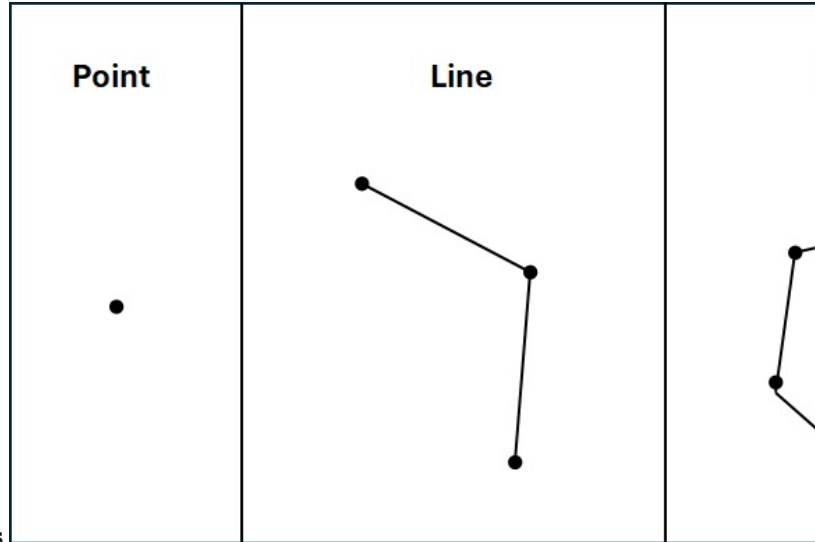


Figure 3. Point, Line, and Polygon Shapes

The above three shapes in Figure 3 consist of points and lines. The points are called vertices. The location of each vertex can be represented by a latitude-longitude coordinate pair. By having shape information and vertex coordinate information, we can identify the geospatial location of a feature on a latitude-longitude coordinate system. In vector data, we store the shape information and coordinate information in a variable called **geometry**. It is this geometry variable that differentiates geospatial data from traditional tabular data. In terms of vector geospatial data format, there are two common data formats: **GeoJSON** and **Shapefile**. The required readings will give a very short introduction to these two data formats. # 6. **Geospatial Data Source and Analysis Software** There are several ways to collect geospatial data. Some of the sources that collect data include the Global Positioning System (GPS), wearable devices, mobile phones, remote sensing (satellite), and internet of things (IoT) to name a few. After getting geospatial data, researchers usually use a **geographic information system (GIS)** to read and analyze the data. There are many popular paid or free GISs, like ArcGIS or QGIS. These types of software usually have built-in functions to read different types of geospatial data and visualize functions to present the data.

2 7. Application of Geospatial Data

In this section, we are going to use Python to demonstrate how to pull geospatial data from online data resource, clean the data to be ready for analysis and visualize the data. We are going to use Hurricane Irene's path along the east coast of the United States as an example. In late August of 2011, Hurricane Irene caused severe damage to this region of the United States. For example, lower New York City lost electricity for one week as a result of the storm.

In this application, we will pull the hurricane's path and wind data from the National Hurricane Center (NHC) of the National Oceanic and Atmospheric Administration (NOAA). We will pull the data and draw the hurricane's path on a map. In this demonstration, we are also going to illustrate how to overlay different information on a map. We will overlay U.S. state borders on the map to show which states were affected by this hurricane. The main Python package to use for geospatial data analysis is geopandas. Geopandas is the geospatial data version of pandas. It can handle the various types of geospatial data we mentioned in previous sections. We then will use the folium package to visualize the hurricane's path. Let's get started.

2.1 7.1 Load Python Packages

There are a few packages we will need for this demonstration.

```
[30]: !pip install fiona
```

```
Requirement already satisfied: fiona in /usr/local/lib/python3.11/site-packages (1.10.1)
```

```
Requirement already satisfied: attrs>=19.2.0 in /usr/local/lib/python3.11/site-packages (from fiona) (23.1.0)
```

```
Requirement already satisfied: certifi in /usr/local/lib/python3.11/site-packages (from fiona) (2022.12.7)
```

```
Requirement already satisfied: click~=8.0 in /usr/local/lib/python3.11/site-packages (from fiona) (8.1.8)
```

```
Requirement already satisfied: click-plugins>=1.0 in /usr/local/lib/python3.11/site-packages (from fiona) (1.1.1)
```

```
Requirement already satisfied: cligj>=0.5 in /usr/local/lib/python3.11/site-packages (from fiona) (0.7.2)
```

```
WARNING: Running pip as the 'root' user can result in broken permissions
```

```
and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv
```

```
[notice] A new release of pip available: 22.3.1 -> 24.3.1
```

```
[notice] To update, run: pip install --upgrade pip
```

```
[2]: !pip install geopandas
```

```
Collecting geopandas
```

```
  Downloading geopandas-1.0.1-py3-none-any.whl (323 kB)  
323.6/323.6
```

```
kB 6.8 MB/s eta 0:00:00a 0:00:01
```

```
Requirement already satisfied: numpy>=1.22 in /usr/local/lib/python3.11/site-packages (from geopandas) (1.26.2)
```

```
Collecting pyogrio>=0.7.2
```

```
  Downloading pyogrio-0.10.0-cp311-cp311-manylinux_2_28_x86_64.whl (24.1 MB)  
24.1/24.1 MB
```

```
65.5 MB/s eta 0:00:0000:0100:01
```

```
Requirement already satisfied: packaging in /usr/local/lib/python3.11/site-packages (from geopandas) (23.1)
```

```
Requirement already satisfied: pandas>=1.4.0 in /usr/local/lib/python3.11/site-packages (from geopandas) (2.0.0)
```

```
Collecting pyproj>=3.3.0
```

```
  Downloading pyproj-3.7.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (9.5 MB)  
9.5/9.5 MB
```

108.8 MB/s eta 0:00:0000:0100:01

Collecting shapely>=2.0.0

Downloading

shapely-2.0.6-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.5 MB)

2.5/2.5 MB

101.6 MB/s eta 0:00:00

Requirement already satisfied: python-dateutil>=2.8.2 in

/usr/local/lib/python3.11/site-packages (from pandas>=1.4.0->geopandas) (2.8.2)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/site-packages (from pandas>=1.4.0->geopandas) (2023.3.post1)

Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.11/site-packages (from pandas>=1.4.0->geopandas) (2023.3)

Requirement already satisfied: certifi in /usr/local/lib/python3.11/site-packages (from pyogrio>=0.7.2->geopandas) (2022.12.7)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-packages (from python-dateutil>=2.8.2->pandas>=1.4.0->geopandas) (1.16.0)

Installing collected packages: shapely, pyproj, pyogrio, geopandas

Successfully installed geopandas-1.0.1 pyogrio-0.10.0 pyproj-3.7.0 shapely-2.0.6

WARNING: Running pip as the 'root' user can result in broken permissions

and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip

available: 22.3.1 -> 24.3.1

[notice] To update, run:

pip install --upgrade pip

[8]: pip install install-jdk

Collecting install-jdk

Downloading install_jdk-1.1.0-py3-none-any.whl (15 kB)

Installing collected packages: install-jdk

Successfully installed install-jdk-1.1.0

WARNING: Running pip as the 'root' user can result in broken permissions

and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip

available: 22.3.1 -> 24.3.1

[notice] To update, run:

pip install --upgrade pip

Note: you may need to restart the kernel to use updated packages.

```
[9]: import jdk
from jdk.enums import OperatingSystem, Architecture

jdk.install('11', operating_system=OperatingSystem.LINUX)
```

```
[9]: '/root/.jdk/jdk-11.0.25+9'
```

```
[10]: import os
jdk_version = 'jdk-11.0.25+9' #change with your version
os.environ['JAVA_HOME'] = '/root/.jdk/jdk-11.0.25+9'
os.environ['PATH'] = f"{os.environ.get('PATH')}:{os.environ.get('JAVA_HOME')}/
↳bin"
```

```
[3]: # Import packages for this application
# pandas is used to process dataframe and geopandas is used to process
↳geodataframe
# fiona is used to read or write various formats of geospatial data
# urllib is used to pull data on the internet using url
# zipfile is used to unzip a zipped file
import os
import pandas as pd
import geopandas as gpd
import fiona
import urllib.request
import zipfile
```

2.2 7.2 Data for Hurricane Irene

The hurricane path data is in a PDF file. As such, we need to download the tabula package to handle PDF files.

```
[4]: pip install tabula-py
```

Collecting tabula-py

Downloading tabula_py-2.10.0-py3-none-any.whl (12.0 MB)

12.0/12.0 MB

60.5 MB/s eta 0:00:0000:0100:01

Requirement already satisfied: pandas>=0.25.3 in

/usr/local/lib/python3.11/site-packages (from tabula-py) (2.0.0)

Requirement already satisfied: numpy>1.24.4 in /usr/local/lib/python3.11/site-packages (from tabula-py) (1.26.2)

Collecting distro

Downloading distro-1.9.0-py3-none-any.whl (20 kB)

Requirement already satisfied: python-dateutil>=2.8.2 in

/usr/local/lib/python3.11/site-packages (from pandas>=0.25.3->tabula-py) (2.8.2)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/site-packages (from pandas>=0.25.3->tabula-py) (2023.3.post1)

Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.11/site-

```

packages (from pandas>=0.25.3->tabula-py) (2023.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-
packages (from python-dateutil>=2.8.2->pandas>=0.25.3->tabula-py) (1.16.0)
Installing collected packages: distro, tabula-py
Successfully installed distro-1.9.0 tabula-py-2.10.0
WARNING: Running pip as the 'root' user can result in broken permissions
and conflicting behaviour with the system package manager. It is recommended to
use a virtual environment instead: https://pip.pypa.io/warnings/venv

```

```

[notice] A new release of pip
available: 22.3.1 -> 24.3.1
[notice] To update, run:
pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.

```

```

[5]: # Import tabula package for PDF handling
import tabula

```

Next, let's retrieve the PDF file from the National Hurricane Center website.

```

[6]: # Retrieve PDF file from NHC website
url = "https://www.nhc.noaa.gov/data/tcr/AL092011_Irene.pdf"

irene_pdf, _ = urllib.request.urlretrieve(url)

```

Then, we will use a method from the tabula package to convert a PDF file to a csv file.

```

[11]: # Convert PDF file to csv file, and then a pandas dataframe
tabula.convert_into(irene_pdf, "irene.csv", output_format="csv", stream=True,
    <-pages = 9)
irene_1 = pd.read_csv("irene.csv")
irene_1

```

```

Got stderr: Jan 20, 2025 8:23:31 PM
org.apache.pdfbox.pdmodel.font.FileSystemFontProvider loadDiskCache
WARNING: New fonts found, font cache will be re-built
Jan 20, 2025 8:23:31 PM org.apache.pdfbox.pdmodel.font.FileSystemFontProvider
<init>
WARNING: Building on-disk font cache, this may take a while
Jan 20, 2025 8:23:31 PM org.apache.pdfbox.pdmodel.font.FileSystemFontProvider
<init>
WARNING: Finished building on-disk font cache, found 6 fonts
Jan 20, 2025 8:23:31 PM org.apache.pdfbox.pdmodel.font.PDTrueTypeFont <init>
WARNING: Using fallback font 'LiberationSans' for 'TimesNewRomanPSMT'
Jan 20, 2025 8:23:31 PM org.apache.pdfbox.pdmodel.font.PDTrueTypeFont <init>
WARNING: Using fallback font 'LiberationSans' for 'TimesNewRomanPS-BoldMT'
Jan 20, 2025 8:23:33 PM org.apache.pdfbox.pdmodel.font.PDTrueTypeFont <init>
WARNING: Using fallback font 'LiberationSans' for 'TimesNewRomanPSMT'

```

Jan 20, 2025 8:23:33 PM org.apache.pdfbox.pdmodel.font.PDTrueTypeFont <init>
 WARNING: Using fallback font 'LiberationSans' for 'TimesNewRomanPS-BoldMT'

```
[11]:      Date/Time Latitude Longitude Pressure Wind Speed      Unnamed: 5
      0      (UTC)      (°N)      (°W)      (mb)      (kt)      Stage
      1  21 / 0000      15.0      59.0      1006      45 tropical storm
      2  21 / 0600      16.0      60.6      1006      45      "
      3  21 / 1200      16.8      62.2      1005      45      "
      4  21 / 1800      17.5      63.7      999      50      "
      5  22 / 0000      17.9      65.0      993      60      "
      6  22 / 0600      18.2      65.9      990      65 hurricane
      7  22 / 1200      18.9      67.0      989      70      "
      8  22 / 1800      19.3      68.0      988      75      "
      9  23 / 0000      19.7      68.8      981      80      "
     10  23 / 0600      20.1      69.7      978      80      "
     11  23 / 1200      20.4      70.6      978      80      "
     12  23 / 1800      20.7      71.2      977      80      "
     13  24 / 0000      21.0      71.9      969      80      "
     14  24 / 0600      21.3      72.5      965      95      "
     15  24 / 1200      21.9      73.3      957     105      "
     16  24 / 1800      22.7      74.3      954     100      "
     17  25 / 0000      23.5      75.1      952      95      "
     18  25 / 0600      24.1      75.9      950      95      "
     19  25 / 1200      25.4      76.6      950      90      "
     20  25 / 1800      26.5      77.2      950      90      "
     21  26 / 0000      27.7      77.3      946      90      "
     22  26 / 0600      28.8      77.3      942      90      "
     23  26 / 1200      30.0      77.4      947      85      "
     24  26 / 1800      31.1      77.5      950      80      "
     25  27 / 0000      32.1      77.1      952      75      "
     26  27 / 0600      33.4      76.8      952      75      "
     27  27 / 1200      34.7      76.6      952      75      "
     28  27 / 1800      35.5      76.3      950      65      "
     29  28 / 0000      36.7      75.7      951      65      "
     30  28 / 0600      38.1      75.0      958      65      "
     31  28 / 0935      39.4      74.4      959      60 tropical storm
     32  28 / 1200      40.3      74.1      963      55      "
     33  28 / 1300      40.6      74.0      965      55      "
     34  28 / 1800      42.5      73.1      970      50      "
     35  29 / 0000      44.2      72.1      979      45 extratropical
     36  29 / 0600      46.5      69.5      983      40      "
     37  29 / 1200      49.1      66.7      985      40      "
     38  29 / 1800      51.3      63.8      987      40      "
     39  30 / 0000      53.0      60.0      991      40      "
     40  30 / 0600      NaN      NaN      NaN      NaN absorbed
```

From the last code output, we can see that the second row does not contain data values. They

are measurement units/information for each variable. For example, the measurement unit for wind speed is knots (kt). Also, the last row does not have any data values. We are going to drop these two rows.

```
[12]: # Drop rows with measurement units or no data values
irene_1.drop([0,40], inplace = True)
irene_1
```

```
[12]:
```

	Date/Time	Latitude	Longitude	Pressure	Wind Speed	Unnamed: 5
1	21 / 0000	15.0	59.0	1006	45	tropical storm
2	21 / 0600	16.0	60.6	1006	45	"
3	21 / 1200	16.8	62.2	1005	45	"
4	21 / 1800	17.5	63.7	999	50	"
5	22 / 0000	17.9	65.0	993	60	"
6	22 / 0600	18.2	65.9	990	65	hurricane
7	22 / 1200	18.9	67.0	989	70	"
8	22 / 1800	19.3	68.0	988	75	"
9	23 / 0000	19.7	68.8	981	80	"
10	23 / 0600	20.1	69.7	978	80	"
11	23 / 1200	20.4	70.6	978	80	"
12	23 / 1800	20.7	71.2	977	80	"
13	24 / 0000	21.0	71.9	969	80	"
14	24 / 0600	21.3	72.5	965	95	"
15	24 / 1200	21.9	73.3	957	105	"
16	24 / 1800	22.7	74.3	954	100	"
17	25 / 0000	23.5	75.1	952	95	"
18	25 / 0600	24.1	75.9	950	95	"
19	25 / 1200	25.4	76.6	950	90	"
20	25 / 1800	26.5	77.2	950	90	"
21	26 / 0000	27.7	77.3	946	90	"
22	26 / 0600	28.8	77.3	942	90	"
23	26 / 1200	30.0	77.4	947	85	"
24	26 / 1800	31.1	77.5	950	80	"
25	27 / 0000	32.1	77.1	952	75	"
26	27 / 0600	33.4	76.8	952	75	"
27	27 / 1200	34.7	76.6	952	75	"
28	27 / 1800	35.5	76.3	950	65	"
29	28 / 0000	36.7	75.7	951	65	"
30	28 / 0600	38.1	75.0	958	65	"
31	28 / 0935	39.4	74.4	959	60	tropical storm
32	28 / 1200	40.3	74.1	963	55	"
33	28 / 1300	40.6	74.0	965	55	"
34	28 / 1800	42.5	73.1	970	50	"
35	29 / 0000	44.2	72.1	979	45	extratropical
36	29 / 0600	46.5	69.5	983	40	"
37	29 / 1200	49.1	66.7	985	40	"
38	29 / 1800	51.3	63.8	987	40	"

```
39 30 / 0000      53.0      60.0      991      40      "
```

Next, let's convert numeric variables to float types.

```
[13]: # Correct data types for numeric variables
convert_dict = {'Date/Time': str,
                'Latitude': float,
                'Longitude': float,
                'Pressure': float,
                'Wind Speed': float
                }

irene_1 = irene_1.astype(convert_dict)
print(irene_1.dtypes)
```

```
Date/Time      object
Latitude       float64
Longitude       float64
Pressure        float64
Wind Speed      float64
Unnamed: 5      object
dtype: object
```

2.3 7.3 Creating the Date-Time Variable

Next, we can see that the date/time variable does not contain month and year information. Therefore, we're going to create a new date/time variable to provide complete date/time information.

```
[14]: # Create a new Date Time variable to contain month and year information
irene_1[["Date", "Time"]] = irene_1["Date/Time"].str.split(" / ", expand = True)
irene_1["Date_Time"] = "08/" + irene_1["Date"] + "/2011/" + irene_1["Time"]
irene_1.set_index("Date_Time")
irene_1
```

```
[14]:
```

	Date/Time	Latitude	Longitude	Pressure	Wind Speed	Unnamed: 5	Date
1	21 / 0000	15.0	59.0	1006.0	45.0	tropical storm	21 \
2	21 / 0600	16.0	60.6	1006.0	45.0	"	21
3	21 / 1200	16.8	62.2	1005.0	45.0	"	21
4	21 / 1800	17.5	63.7	999.0	50.0	"	21
5	22 / 0000	17.9	65.0	993.0	60.0	"	22
6	22 / 0600	18.2	65.9	990.0	65.0	hurricane	22
7	22 / 1200	18.9	67.0	989.0	70.0	"	22
8	22 / 1800	19.3	68.0	988.0	75.0	"	22
9	23 / 0000	19.7	68.8	981.0	80.0	"	23
10	23 / 0600	20.1	69.7	978.0	80.0	"	23
11	23 / 1200	20.4	70.6	978.0	80.0	"	23
12	23 / 1800	20.7	71.2	977.0	80.0	"	23
13	24 / 0000	21.0	71.9	969.0	80.0	"	24
14	24 / 0600	21.3	72.5	965.0	95.0	"	24

15	24 / 1200	21.9	73.3	957.0	105.0	"	24
16	24 / 1800	22.7	74.3	954.0	100.0	"	24
17	25 / 0000	23.5	75.1	952.0	95.0	"	25
18	25 / 0600	24.1	75.9	950.0	95.0	"	25
19	25 / 1200	25.4	76.6	950.0	90.0	"	25
20	25 / 1800	26.5	77.2	950.0	90.0	"	25
21	26 / 0000	27.7	77.3	946.0	90.0	"	26
22	26 / 0600	28.8	77.3	942.0	90.0	"	26
23	26 / 1200	30.0	77.4	947.0	85.0	"	26
24	26 / 1800	31.1	77.5	950.0	80.0	"	26
25	27 / 0000	32.1	77.1	952.0	75.0	"	27
26	27 / 0600	33.4	76.8	952.0	75.0	"	27
27	27 / 1200	34.7	76.6	952.0	75.0	"	27
28	27 / 1800	35.5	76.3	950.0	65.0	"	27
29	28 / 0000	36.7	75.7	951.0	65.0	"	28
30	28 / 0600	38.1	75.0	958.0	65.0	"	28
31	28 / 0935	39.4	74.4	959.0	60.0	tropical storm	28
32	28 / 1200	40.3	74.1	963.0	55.0	"	28
33	28 / 1300	40.6	74.0	965.0	55.0	"	28
34	28 / 1800	42.5	73.1	970.0	50.0	"	28
35	29 / 0000	44.2	72.1	979.0	45.0	extratropical	29
36	29 / 0600	46.5	69.5	983.0	40.0	"	29
37	29 / 1200	49.1	66.7	985.0	40.0	"	29
38	29 / 1800	51.3	63.8	987.0	40.0	"	29
39	30 / 0000	53.0	60.0	991.0	40.0	"	30

	Time	Date_Time
1	0000	08/21/2011/0000
2	0600	08/21/2011/0600
3	1200	08/21/2011/1200
4	1800	08/21/2011/1800
5	0000	08/22/2011/0000
6	0600	08/22/2011/0600
7	1200	08/22/2011/1200
8	1800	08/22/2011/1800
9	0000	08/23/2011/0000
10	0600	08/23/2011/0600
11	1200	08/23/2011/1200
12	1800	08/23/2011/1800
13	0000	08/24/2011/0000
14	0600	08/24/2011/0600
15	1200	08/24/2011/1200
16	1800	08/24/2011/1800
17	0000	08/25/2011/0000
18	0600	08/25/2011/0600
19	1200	08/25/2011/1200
20	1800	08/25/2011/1800

```

21 0000 08/26/2011/0000
22 0600 08/26/2011/0600
23 1200 08/26/2011/1200
24 1800 08/26/2011/1800
25 0000 08/27/2011/0000
26 0600 08/27/2011/0600
27 1200 08/27/2011/1200
28 1800 08/27/2011/1800
29 0000 08/28/2011/0000
30 0600 08/28/2011/0600
31 0935 08/28/2011/0935
32 1200 08/28/2011/1200
33 1300 08/28/2011/1300
34 1800 08/28/2011/1800
35 0000 08/29/2011/0000
36 0600 08/29/2011/0600
37 1200 08/29/2011/1200
38 1800 08/29/2011/1800
39 0000 08/30/2011/0000

```

2.4 7.4 Ensuring Spatial Data is Correct

We also notice that the data values for longitude in the dataset are all positive. However, the direction for longitude is “W.” The numbers for longitude should all be negative based on Figure 1 in section 3. Hence, we need to add a negative sign in front of all numbers for the longitude variable to correctly reflect the location.

```

[15]: # Adjust Longitude value to correctly reflect the geolocation
      irene_1['Longitude'] = 0 - irene_1['Longitude']

```

```

[16]: # Select the variables we are interest for next steps
      irene_2 = irene_1[["Date_Time", "Longitude", "Latitude", "Wind Speed"]]

```

2.5 7.5 Converting to a Geodataframe

Now we have a good dataframe for Hurricane Irene’s path. To convert this dataframe to a geospatial dataframe, we need to create a geometry variable, as explained in the previous section. We will use a method from geopandas to create this variable. One of the parameters in the following code is “EPSG:4326”, which is the code name for the latitude-longitude system we are familiar with. Remember there are several CRS systems available, but we’ll proceed with this commonly used one.

```

[17]: # Create a geolocation variable
      geometry = gpd.points_from_xy(irene_2.Longitude, irene_2.Latitude, crs="EPSG:
      ↪4326")

```

Now let’s convert the current pandas dataframe to a geodataframe.

```
[18]: # Convert the current dataframe to a geodataframe with geometry variable
irene_3 = gpd.GeoDataFrame(
    irene_2, geometry=geometry, crs="EPSG:4326"
)
```

Great! Now we have a geodataframe. Let's check out the first five entries in this dataframe.

```
[19]: irene_3.head()
```

```
[19]:
```

	Date_Time	Longitude	Latitude	Wind Speed	geometry
1	08/21/2011/0000	-59.0	15.0	45.0	POINT (-59 15)
2	08/21/2011/0600	-60.6	16.0	45.0	POINT (-60.6 16)
3	08/21/2011/1200	-62.2	16.8	45.0	POINT (-62.2 16.8)
4	08/21/2011/1800	-63.7	17.5	50.0	POINT (-63.7 17.5)
5	08/22/2011/0000	-65.0	17.9	60.0	POINT (-65 17.9)

Under the geometry variable in the above geodataframe, we have point shapes and their coordinate pairs on the map. Let's confirm the type of our new dataframe.

```
[20]: type(irene_3)
```

```
[20]: geopandas.geodataframe.GeoDataFrame
```

And let's check our geometry variable.

```
[21]: type(irene_3['geometry'])
```

```
[21]: geopandas.geoseries.GeoSeries
```

The geodataframe basically behaves like the pandas dataframe. Therefore, we can apply the same methods and analysis from pandas to geopandas. Here is one example.

```
[22]: print("Mean wind speed of Hurricane Irene is {} knots and it can go up to {}_
    ↪knots maximum".format(round(irene_2['Wind Speed'].mean(),4),
    ↪
    ↪       irene_2['Wind Speed'].max())+".")
```

Mean wind speed of Hurricane Irene is 69.6154 knots and it can go up to 105.0 knots maximum.

2.5.1 7.6 Data for U.S. State Borders

Before we draw Hurricane Irene's path on a map, we would like to add a layer with U.S. state borders to the map. With the state border visualization along with Hurricane Irene's path, we can learn which states were affected by this hurricane. We will pull the state border file from the United States Census Bureau's website. This is a zipped shapefile. We first need to import a package to unzip the file.

```
[23]: # Import a file unzip package
from zipfile import ZipFile
```

```
[24]: # Retrieve US State Shapefile from United States Census Bureau
us_state_url = "http://www2.census.gov/geo/tiger/TIGER2012/STATE/
            ↪tl_2012_us_state.zip"

us_state_shape_zip, _ = urllib.request.urlretrieve(us_state_url)
```

```
[25]: # Unzip the zipped shapefile and assign it a new file name "us_state_shape"
with ZipFile(us_state_shape_zip, 'r') as zObject:

    zObject.extractall("us_state_shape")
```

Once we unzip the file, we will use the `read_file` method from `geopandas` to read this file as a `geodataframe` for further data processing.

```
[26]: # Read in our shapefile to a geodataframe
us_state_shape_g = gpd.read_file("us_state_shape")
```

```
[27]: # Check the metadata of the new geodataframe
us_state_shape_g.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 56 entries, 0 to 55
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   REGION      56 non-null    object
1   DIVISION    56 non-null    object
2   STATEFP     56 non-null    object
3   STATENS     56 non-null    object
4   GEOID       56 non-null    object
5   STUSPS      56 non-null    object
6   NAME        56 non-null    object
7   LSAD        56 non-null    object
8   MTFCC       56 non-null    object
9   FUNCSTAT    56 non-null    object
10  ALAND        56 non-null    int64
11  AWATER       56 non-null    int64
12  INTPTLAT    56 non-null    object
13  INTPTLON    56 non-null    object
14  geometry    56 non-null    geometry
dtypes: geometry(1), int64(2), object(12)
memory usage: 6.7+ KB
```

```
[28]: # Check a few entries of the new geodataframe
us_state_shape_g.head()
```

```
[28]:  REGION DIVISION STATEFP  STATENS GEOID STUSPS  NAME LSAD MTFCC
0      4          9      15  01779782   15    HI   Hawaii  00  G4000  \
```

1	3	7	05	00068085	05	AR	Arkansas	00	G4000
2	4	8	35	00897535	35	NM	New Mexico	00	G4000
3	4	8	30	00767982	30	MT	Montana	00	G4000
4	1	2	36	01779796	36	NY	New York	00	G4000

	FUNCSTAT	ALAND	AWATER	INTPTLAT	INTPTLON
0	A	16634247483	11678744699	+19.8097670	-155.5061027 \
1	A	134772564356	2959210006	+34.8955256	-092.4446262
2	A	314161109357	756438507	+34.4346843	-106.1316181
3	A	376963571188	3868564895	+47.0511771	-109.6348174
4	A	122057936950	19238848209	+42.9133974	-075.5962723

```

                                geometry
0 MULTIPOLYGON (((-155.96333 19.08159, -155.9634...
1 POLYGON ((-94.46025 34.53838, -94.46026 34.543...
2 POLYGON ((-109.04616 34.57929, -109.04616 34.5...
3 POLYGON ((-114.33289 46.66076, -114.33367 46.6...
4 MULTIPOLYGON (((-79.64546 41.99886, -79.6498 4...
```

2.5.2 7.7 Geospatial Data Visualization

Now we have a file for Hurricane Irene's path and a file for U.S. state borders. They are also both in geodataframe forms. We can put all the information in one map for visualization. We will use the folium package to draw the map and add the U.S. state borders and the hurricane's path to the map.

```
[31]: pip install folium
```

```

Collecting folium
  Downloading folium-0.19.4-py2.py3-none-any.whl (110 kB)
    110.5/110.5
kB 3.6 MB/s eta 0:00:00
Collecting branca>=0.6.0
  Downloading branca-0.8.1-py3-none-any.whl (26 kB)
Requirement already satisfied: jinja2>=2.9 in /usr/local/lib/python3.11/site-
packages (from folium) (3.1.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/site-packages
(from folium) (1.26.2)
Requirement already satisfied: requests in /usr/local/lib/python3.11/site-
packages (from folium) (2.29.0)
Collecting xyzservices
  Downloading xyzservices-2025.1.0-py3-none-any.whl (88 kB)
    88.4/88.4 kB
8.5 MB/s eta 0:00:00
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.11/site-packages (from jinja2>=2.9->folium) (2.1.2)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/site-packages (from requests->folium) (3.1.0)
```

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-packages (from requests->folium) (3.4)

Requirement already satisfied: urllib3<1.27,>=1.21.1 in /usr/local/lib/python3.11/site-packages (from requests->folium) (1.26.15)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/site-packages (from requests->folium) (2022.12.7)

Installing collected packages: xyzservices, branca, folium

Successfully installed branca-0.8.1 folium-0.19.4 xyzservices-2025.1.0

WARNING: Running pip as the 'root' user can result in broken permissions

and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip

available: 22.3.1 -> 24.3.1

[notice] To update, run:

pip install --upgrade pip

Note: you may need to restart the kernel to use updated packages.

```
[32]: # Import a mapping library
import folium
```

Great! Now it's time to put the information on a map.

```
[33]: # Draw Hurricane Irene's path and other information to a map

# First, create a basemap
map = folium.Map(location=[30,-102], zoom_start=4, control_scale=True)

# Then add the first layer of US state borders to the map
folium.GeoJson(us_state_shape_g).add_to(map)

# Then add the hurricane travel path to the map. We use a red dot to represent
↳ the hurricane's location at a specific date/time. Then we add an information
↳ box and a popup box. If you hover your mouse cursor to the red dot, the map
↳ will show you date/time linked to the location and the wind speed.
folium.GeoJson(irene_3,
                marker=folium.Circle(radius=2000, fill_color="red",
↳ fill_opacity=0.4, color="red", weight=5),
                tooltip=folium.GeoJsonTooltip(fields=["Date_Time", "Wind Speed"]),
                popup=folium.GeoJsonPopup(fields=["Date_Time", "Wind Speed"]),
↳ add_to(map)

map
```

```
[33]: <folium.folium.Map at 0x7a64d01b7dd0>
```

Voila! We just created a map overlaid with U.S. state borders and Hurricane Irene's path. In

the upper left corner of the map, there is an icon you can use to zoom in and out. We see U.S. state borders in solid blue lines on the map. Hurricane Irene's path is represented by a series of red dots on the map. When you hover your cursor over one of the red dots, an information box will show up, providing date/time and wind speed information. With this map, we can see Hurricane Irene went through most of the northern states along the east coast of the U.S. Wind speed was at its strongest when the hurricane was passing through between the Dominican Republic and the Bahamas. The wind speed slowed down significantly after landfall.

2.6 8. Conclusion

In this lesson, we first explained what geospatial data is. We introduced the concept of using a coordinate reference system (CRS) for providing geoposition information on Earth's surface. We also explained vector data and its common data formats. Then, we demonstrated a simple geospatial data application using Python. Through this application, we learned how to process different types of geospatial data. We also learned how to overlay processed data onto a map for data visualization.

2.6.1 References

1. Awati, Rahul. "Latitude and Longitude." Informa TechTarget, August 2022. <https://www.techtarget.com/whatis/definition/latitude-and-longitude>.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. NASA. "Remote Sensing." *NASA EarthData*. <https://www.earthdata.nasa.gov/learn/backgrounders/remote-sensing#spectral-resolution>

- **Required pages:** Whole webpage
- **Estimated reading time:** 30 minutes

2. NASA Goddard. "9 Things about Landsat 9." *YouTube*, uploaded 26 Sept. 2021, https://www.youtube.com/watch?v=DGE-N8_LQBo.

- **Required pages:** Whole video
- **Estimated reading time:** 6 minutes

Satellite Imagery

MODULE 6 | LESSON 2

Required Readings

Required Readings for this program are open access, which means you can access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. NASA. "Remote Sensing." *NASA EarthData*.

<https://www.earthdata.nasa.gov/learn/backgrounders/remote-sensing#spectral-resolution>

- **Required pages:** Whole webpage
- **Estimated reading time:** 30 minutes

2. NASA Goddard. "9 Things about Landsat 9." *YouTube*, uploaded 26 Sept. 2021,

https://www.youtube.com/watch?v=DGE-N8_LQBo.

- **Required pages:** Whole video
- **Estimated reading time:** 6 minutes

MODULE 6 | LESSON 2

Satellite Imagery

Reading Time	60 minutes
---------------------	------------

Prior Knowledge	Basic Python
------------------------	--------------

|**Keywords** |Remote sensing, Electromagnetic spectrum, Spatial resolution | | |

1. Introduction

In this lesson, we are going to introduce a type of data that we have not touched upon yet: **satellite imagery**. Traditionally, satellite images are the type of data used in natural sciences or the defense sector. However, in recent years, more and more financial analysts have been using satellite images to conduct analyses, hoping to gain extra insights on top of traditional financial analysis to make better financial decisions. The content in this lesson draws from the information in your required readings. We'll first introduce the basics of satellite imagery.

Then, we will briefly talk about some applications of satellite images in financial analysis. Finally, we will use Google Earth Engine (GEE) to access some satellite images and provide examples of how we can use satellite images for analysis.

2. Financial Application of Satellite Imagery

In this section, we are going to talk about how to incorporate satellite images as part of data for financial analysis.

One of the most famous cases of using satellite images for financial analysis is to use satellite images to count the number of cars in and out of Walmart stores' parking lots over a period of time. Financial analysts use this piece of information as a proxy of foot traffic to gauge the sales of the store during a specific period of time. This leads to the first application of using satellite images.

2.1 Monitoring Economic Activities of a Location

The Walmart parking lot example is a classic application of satellite imagery. Another example is to use satellite images to monitor container ship traffic at a major canal or a major port to identify possible supply chain obstacles. We can also use satellite images to monitor factory sites to evaluate potential production levels of a certain time. We can also monitor a mining site to track commodity production.

2.2 Tracking Crop Yield

We can use satellite images to study if a given crop, like wheat or corn, is healthy or not. From this information, financial analysts can estimate if the market is going to have an oversupply of a crop or a shortage of a crop and trade on this information. We usually would use a combination of infrared bands to conduct this analysis.

2.3 Investigating Damages of Natural Disasters

Insurance companies now use satellite images to assess damages from a flood or a wildfire. This method of gathering damage information is more effective than traditional methods of visiting the damage sites. First, the traditional method may require several months to gather data, but using satellite images only takes a few hours. Secondly, many damage sites are usually not accessible after a disaster, so the traditional method is not possible. With this new satellite image technology, insurance companies are able to process insurance claims faster and in more cost-effective ways.

2.4 Constraints of Using Satellite Imagery for Financial Analysis

Currently, for the general public, there are a few issues when it comes to using satellite images for investment analysis.

(A) The temporal resolutions for public available satellite images can be long. For example, Landsat 9 has a temporal resolution of 8 days. Many commercial satellites offer satellite images with short temporal resolution, like a day. However, they are expensive.

(B) Processing large amount of satellite images requires powerful computing infrastructure, which individuals usually don't have.

(C) Processing raw satellite images and converting them into usable images requires special training.

Due to these constraints in using satellite images for analysis, there are many companies that now offer service to access their analytical platforms for satellite images with a subscription. These companies collect public and private satellite images, process them, and then upload them to their platforms. Their platforms generally incorporate machine learning capabilities for analysis. However, the fees to access these platforms are high, so only hedge funds or big investment companies can access them. Take the example of using retail parking lot car traffic as an indicator for stock trading. It is still a profitable trading strategy, but only hedge funds will have access to this information.

3. The Basics of Satellite Imagery

3.1 Remote Sensing

Before talking about satellite images, we need to introduce a few concepts. The first is remote sensing. **Remote sensing** is the activity of using Earth observation satellites or airplanes equipped with sensors to obtain images and other information about the Earth's surface from above.

The sensors on a satellite measure **electromagnetic radiation (EMR)**, which is first emitted by the sun. Once it reaches Earth's surface, it is then reflected back to the sensors on the satellite. Figure 1 below demonstrates the path of EMR movement.

Figure 1: Illustration of How a Satellite Measures EMR Reflected from Earth's Surface



Adapted from: NASA EarthData. "What Is Remote Sensing?" 10 December 2024.

<https://www.earthdata.nasa.gov/learn/backgrounders/remote-sensing>.

3.2 Electromagnetic Spectrum

EMR is a type of energy that travels in waves. However, these waves are not homogeneous; they have different wavelengths. The wavelength of a wave is measured as the distance between two wave tops. If one wave has a shorter wavelength than the other waves, then this wave has a higher frequency because the wave moves up and down and then up again faster.

We group waves with similar wavelengths into a band called a **spectrum**. Figure 2 below shows how we group waves into different spectrums or bands by their wavelengths.

Figure 2: Electromagnetic Radiation Spectrum alt text

Adapted from: NASA EarthData. "What Is Remote Sensing?" 10 December 2024.

<https://www.earthdata.nasa.gov/learn/backgrounders/remote-sensing>.

Figure 2 represents the whole spectrum of electromagnetic radiation. However, in general, Earth observation satellites don't collect data across the whole spectrum. Satellites usually only collect information from infrared bands and bands visible to the human eye. The information from these bands provides enough data for us to investigate activities happening on Earth's surface.

All electromagnetic radiation is light, but only certain wavelengths can be detected by the human eye. This is known as the visible light spectrum. All the colors that humans can see are combinations of three primary colors: red, green, and blue (RGB). Each of these primary colors represents a band.

Photos taken from cameras are also a combination of these three primary colors. These combinations can reveal different levels of information on Earth, and they are very insightful for researchers. We will only use a few combinations in this lesson. For those who are interested in learning more about combinations, there is plenty of information on the internet.

Satellites that collect information from multiple bands are called multispectral satellites. Each satellite can construct its own band system. For example, a popular satellite for which the public can access satellite images for free is the United States' Landsat satellites. Figure 3 below shows the band structure for the U.S. Landsat 8 satellite.

Figure 3: Band Structure for U.S. Landsat 8 Satellite alt text

Adapted from: USGS. "What Are the Band Designations for the Landsat Satellites?" 31 May 2024. <https://www.usgs.gov/faqs/what-are-band-designations-landsat-satellites>

According to Figure 3, if we want to generate a satellite image that looks like a photo taken from a camera, we will need to extract images from band 4 (red), band 3 (green), and band 2

(blue) and combine them together into a regular photo-like image. Figure 4 below demonstrates this concept.

Figure 4: Color Bands For Photo-Like Image  alt text

Adapted from: Humboldt State Geospatial Online. "Natural and False Color Composites." 2014. https://gsp.humboldt.edu/olm/Courses/GSP_216/lessons/composites.html

3.3 Spatial Resolution

When a satellite collects information from an observation area, it divides the area into a matrix or a grid. Each cell in the matrix is called a **pixel**. A cell is a square. If the cell is square that is 30 meters x 30 meters, then the resolution is 30 meters. The number represents the size of a pixel. Figure 5 demonstrates this concept.

Figure 5: Spatial Resolution  alt text

From Figure 5, we can see that if the size of a pixel is smaller, say 10 meters instead of 30 meters, more pixels can fit into one area. This means that the image will have a higher resolution. In a pixel, the satellite will measure the EMR intensity of the band where the pixel is located and assign a number to the pixel. If the digital images of a band from a satellite have higher resolutions, it means there are more pixels in the images and more data in the images as well. When we read an image with higher resolution, the image will look more refined, and the lines and curves will be smooth. When we read an image with lower resolution, the image will look pixelated or more like a mosaic. Figure 6 demonstrates this difference.

Figure 6: Images of the Same Location with Different Resolutions  alt text

From Figure 6, we can see the photo on the left has a resolution of 30 meters. We can observe a lot of details of the location from this photo, like roads and properties. However, the photo on the right has a resolution of 300 meters. The photo looks like a mosaic—we can only see the vague shape of the location and can only identify different color blocks of geographical features like water and land.

Let's go back to Figure 3 to check the resolution construct for satellite Landsat 8. The right column of the figure shows the resolution setup for each band on Landsat 8. Resolutions for most bands are 30 meters. However, Band 8 has a resolution of 15 meters, whereas Band 10 and Band 11 have resolutions of 100 meters. From these, we can see different bands can have different resolutions within a satellite.

3.4 Popular Satellites to Provide Images

The Landsat series from the U.S. and Sentinel 2 from the European Space Agency are the two most popular satellite systems that provide satellite images for research. The most current satellite for the Landsat series is Landsat 9. There are other Landsat satellites in operation as well, like Landsat 8 and Landsat 7. Sentinel 2 has a pair of satellites working currently.

Satellites orbit the Earth to collect data. Because they orbit the Earth at a different speed than the speed of Earth's rotation, they don't collect data from the same spot every day. Landsat 8 and Landsat 9 combined collect information from the same spot every 8 days. The pair of Sentinel 2 satellites collect information from the same spot every 5 days. The number of days it takes a satellite to collect information from the same spot multiple times is called **temporal resolution**.

Please watch the video from the Required Readings list to get more information about Landsat series satellites.

3.5 Summary of the Basics of Satellite Imagery

Let's summarize what we've learned so far. We know that a satellite records reflected energy of different wavelengths based on its band structure from Earth's surface. For each band, the collected data will be stored in a pixel of a matrix of the observation area at a certain time and be created as a digital image of the observation area. Digital images of different bands can be combined for the purposes of research and analysis. Therefore, it is important to know which band or combination of bands provides appropriate images for your research. If your research focuses on identifying objects on Earth's surface, you will also need high-resolution images for this task.

Now we have a basic understanding of satellite imagery. In the next section, we will briefly talk about current trends in using satellite imagery to conduct financial analysis.

4. Satellite Imagery Application: Google Earth Engine

In this section, we are going to use the Google Earth Engine (GEE) platform to retrieve satellite images and do some analysis. Google Earth Engine is a platform that we can use to conduct analysis for satellite images and other geospatial data. The GEE team has collected and processed publicly available historical satellite images from popular satellites and stores them in its Earth Engine data catalog. Hence, we don't need to spend time and effort to collect and process raw historical satellite images. GEE has a code editor user interface that allows users to retrieve and analyze satellite images. It also has a Python API. In this lesson, we will use the Python API to retrieve images and conduct analysis. We will use the video from UCLA Office of Research Computing (UCLA 2022) to demonstrate this application.

4.1 Scenario: 2018 Northern California Camp Fire

On Thursday, November 8, 2018, a faulty electric transmission line caught fire in Butte County, California, in the United States. Due to strong downslope wind, the fire spread quickly and burned around 153,000 acres of land. It did not stop until November 25, 2018. The Camp Fire was the most expensive natural disaster in 2018. In this section, we are going to look at the satellite images before the fire, during the fire, and after the fire. We will also do one analysis to investigate the vegetation conditions before and after the fire.

4.2 Set Up Google Earth Engine

Before using Google Earth Engine, please read the file linked below. This file outlines how to set up a Google Earth Engine account step by step.

https://docs.google.com/document/d/1xZwsOHp49aCQFsPzvP0Kd8SJGmwMZA9Q/edit?usp=drive_link&ouid=103990088807346702419&rtpof=true&sd=true

5. Python API Demonstration on Google Colab

5.1 Load the Necessary Libraries for This Demonstration#

We will use Google Colab to access Python API for GEE for this demonstration.

```
In [ ]: # pandas to handle tabular data and geopandas to handle geospatial data
import pandas as pd
import geopandas as gpd

# earth engine
import ee

# Google earth engine map
import geemap

# allow images to display in the notebook
from IPython.display import Image
```

5.2 Access to Google Earth Engine###

We will use the following code to access Google Earth Engine's API. Please replace 'ee-si-learning-001' with your own project ID. Your Project ID was created when you registered for Google Earth Engine. During the authentication process, a dialogue box will appear. It will

ask you to select the Google account you used to register for Google Earth Engine. Please click that account. Then, there will be some information along with a "continue" button on the lower right side of the dialogue box. Please click the "continue" button. Then, there will be another message, and you will need to go through and hit the "continue" button to finish the authentication process. There will be two continue buttons to hit in total.

```
In [ ]: # Start Google Earth Engine authentication process
ee.Authenticate()

# Initialize Google Earth Engine with the project ID you set up during the account
ee.Initialize(project='[REPLACE PROJECT ID]')
```

5.3 Set Up Filter Parameters###

Define the location of interest using latitude and longitude coordinates as well as a start and end period to pull satellite images. We can use Google Maps to find latitude and longitude coordinates for Butte County in California. Since the fire started on November 8, 2018, we will pull the images from October 2018 to December 2018.

```
In [ ]: # coordinates of the Camp Fire
lat = 39.444012
lon = -121.833619

# point of interest as an ee.Geometry
poi = ee.Geometry.Point(lon,lat)

# start date of range to filter for
start_date = '2018-10-01'

# end date
end_date = '2019-01-31'
```

5.4 Retrieve Images from Google Earth Engine Data Catalog

We will retrieve the images from Landsat 8. The following link from Google Earth Engine provides information about this image collection from Landsat 8. It also provides the Python code snippet to pull the images shown in the following code.

https://developers.google.com/earth-engine/datasets/catalog/LANDSAT_LC08_C02_T1_L2

```
In [ ]: # get the satellite data
landsat = ee.ImageCollection("LANDSAT/LC08/C02/T1_L2")\
    .filterBounds(poi)\
    .filterDate(start_date,end_date)
```

5.5 Check the Image Information

Now we have downloaded the images. Let's review some information. First, let's take a look at how many images we got from the selection period. Landsat 8's temporal resolution is 16 days. Our selection period is 3 months. Hence, we should get at least 6 images. $((3*30)/16 = 5.6)$

```
In [ ]: # Check how many images we get during the selection period.
print('Total number:', landsat.size().getInfo())
```

Next, we use the first image we pulled to see what information we can get.

```
In [ ]: landsat.first().getInfo()
```

One key element in satellite imagery is how much of the image is covered by clouds. An image with heavy cloud coverage will not provide much information for our analysis. We usually would like the cloud coverage of an image to be around or below 0.05. Let's take a look the scale of cloud coverage in the first image.

```
In [ ]: landsat.first().get('CLOUD_COVER').getInfo()
```

We can see the cloud coverage of the first image is 0.05, which is good. Now let's check when the first image was taken.

```
In [ ]: landsat.first().get('DATE_ACQUIRED').getInfo()
```

The next thing we can check is the band names we get.

```
In [ ]: landsat.first().bandNames().getInfo()
```

We see from the above code results that there are more than 11 bands we learned from the previous section. The band names with 'ST_' are all under one band. They are sub-bands for the Aerosol band.

5.6 Visualize Images

We are now ready to see the images we pulled. First, let's create labels for images to be used in the following visualization.

```
In [ ]: # put the images in a list
landsat_list = landsat.toList(landsat.size());

#Create labels for images
labels = ["Image #" + str(i) + ", " + str(ee.Image(landsat_list.get(i)).get('DATE_ACQUIRE
labels
```

From the image labels, we can see we have 8 images from image 0 to image 7. We also see the dates they were taken on and their cloud coverage index.

Now let's define some parameters to display the images. We will use red, green, and blue bands to compose an image that resembles a photo. We will also define the pixel dimensions to show in our image. Last, we set the brightness by assigning min and max values. Assign values to min and max is a trial-and-error process. You need to play around with the number to get the appropriate brightness for images.

```
In [ ]: parameters = {  
        'min': 7000,  
        'max': 16000,  
        'dimensions': 800, # square size in pixels  
        'bands': ['SR_B4', 'SR_B3', 'SR_B2'] # bands to display (r,g,b)  
    }
```

Now let's use geemap to display the first image overlaid on a map.

```
In [ ]: Map = geemap.Map()  
  
first_image = landsat.first()  
  
Map.addLayer(first_image, parameters, "First_Image")  
Map.setCenter(lon,lat,8)  
Map
```

From the above map, you can see the first image from our image collection. This is the image shot on November 7, 2018. You can move your cursor onto the map and move the map around. You can zoom in and out of the image by clicking the "+" or "-" icon on the left side of the map.

We just saw how to display an image on the map. What if we want to interactively display other images in our collection on the map? We will create a new map and add a time series slider on the map to achieve this goal.

```
In [ ]: image = landsat.toBands()  
  
Map2 = geemap.Map()  
Map2.addLayer(image,{}, "Time series", False)  
Map2.setCenter(lon,lat,8)  
Map2.add_time_slider(landsat, parameters, labels = labels, time_interval=1)  
Map2
```

We can see the new map has a slider on the lower right side of the map. **A note on using the slider:** Due to some technical issues, please do not use the "Play the time slider" button.

On the slider, you can see the label of the first image (image #0) by the slider bar. If you move your cursor to the dot on the left of the slider bar, you will see the dot turns blue. You can then move the blue dot along the slider bar to see other images. The label will display the image information you are seeing on the map. For example, in image #3, there is heavy

cloud cover, with the cloud coverage at 67, so most of what we see is white. If we move to image #5, its cloud coverage is 6, so we can get a better look at Earth's surface.

5.7 Select and Zoom in on Cloudless Images

We have investigated all the images we pulled from the data catalog. Now we would like to select the ones we will use for analysis. We would like to select 3 cloudless images: one from before the fire, one during the fire, and one after the fire. We would also like to zoom in on the area we are interested in for each image. By investigating all 8 images, we can see image #0, image #2, and image #5 fit our selection criteria.

```
In [ ]: #Select images we want and create a new image collection
img1 = ee.Image(landsat_list.get(0))
img2 = ee.Image(landsat_list.get(2))
img3 = ee.Image(landsat_list.get(5))

landsat_2 = ee.ImageCollection.fromImages([img1, img2, img3])
print('Total number:', landsat_2.size().getInfo())
```

Now we create new labels for a new collection of images. We also update our new visualization parameters.

```
In [ ]: # Create a new List
landsat_list_2 = landsat_2.toList(landsat_2.size())

# Define a region of interest with a buffer zone
roi = poi.buffer(20000) # meters

# New visualization parameters
parameters_2 = {
    'min': 6000,
    'max': 16000,
    'dimensions': 800,
    'bands': ['SR_B4', 'SR_B3', 'SR_B2'],
    'region':roi
}

#Create new Labels for images
labels_2 = ["Image #"+str(i)+", "+str(ee.Image(landsat_list_2.get(i)).get('DATE_ACQ
labels_2
```

```
In [ ]: image2 = landsat_2.toBands()

Map3 = geemap.Map()
Map3.addLayer(image2,{}, "Time series", False)
Map3.setCenter(lon,lat,8)
Map3.add_time_slider(landsat_2, parameters_2, labels = labels_2, time_interval=1)
Map3
```

We can see that the upper corner of the middle image does show the fire. However, it is still hard to see the impact of the fire when comparing the first image to the last image. Let's introduce an NDVI index to better study the damage.

5.8 Normalized Difference Vegetation Index (NDVI)

The Normalized Difference Vegetation Index (NDVI) is an index to study plant health. This index is calculated by using reflected light from the red band and near infrared band of satellite images. The higher the index number means the plants are healthier and greener. The lower the index number means the plants are browner and less healthy. The concept is illustrated in Figure 7.

Figure 7. NDVI Illustration  alt text

In the following section, we will turn our three images into NDVI images. Each pixel will have an NDVI number. A pixel with a high NDVI number will be painted green. A pixel with a low NDVI number will be painted red. If the NDVI number is in between, the pixel will be painted yellow.

First, let's calculate NDVI numbers for each image.

```
In [ ]: #Calculate NDVI for each image
img_ndvi_1 = ee.Image(landsat_list_2.get(0)).normalizedDifference(['SR_B5', 'SR_B4'])
img_ndvi_2 = ee.Image(landsat_list_2.get(1)).normalizedDifference(['SR_B5', 'SR_B4'])
img_ndvi_3 = ee.Image(landsat_list_2.get(2)).normalizedDifference(['SR_B5', 'SR_B4'])

landsat_3 = ee.ImageCollection.fromImages([img_ndvi_1, img_ndvi_2, img_ndvi_3])
print('Total number:', landsat_3.size().getInfo())
```

Then, we'll set up the new color palette and NDVI visualization parameters.

```
In [ ]: # Create a new List
landsat_list_3 = landsat_3.toList(landsat_3.size())

# ndvi palette: red is low, green is high vegetation
palette = ['red', 'yellow', 'green']

ndvi_parameters = {'min': 0,
                  'max': 0.4,
                  'dimensions': 512,
                  'palette': palette,
                  'region': roi}
```

Now let's display the NDVI images.

```
In [ ]: image3 = landsat_3.toBands()

Map4 = geemap.Map()
Map4.addLayer(image3, {}, "Time series", False)
Map4.setCenter(lon, lat, 8)
Map4.add_time_slider(landsat_3, ndvi_parameters, labels = labels_2, time_interval=1)
Map4
```

From the above interactive NDVI images, let's focus on the upper middle part of each image. Let's start with image #0.

In image #0, the area contains a mixture of green, yellow, and red. This image was taken before the Camp Fire. In image #1, we can see a big swath of red running across the area from right to left and going down. This image was taken during the Camp Fire. The red area is where the fire was. Now let's look at image #3, which was taken after the Camp Fire. In image #3, we can see a big red area in the upper middle part and some on the upper left side of the image. To summarize, with NDVI images, we can better identify changes in the vegetation on Earth's surface.

6. Conclusion

In this lesson, we first learned the basics of remote sensing, the method to obtain satellite images. We then went through the fundamentals of satellite images, including the electromagnetic spectrum, spatial resolution, and commonly used satellites. We then introduced some use cases for applying satellite images to finance analyses. We finished the lesson by using Google Earth Engine's Python API to pull some satellite images and conducted some analysis. We learned to use NDVI images to investigate the vegetation damage after the 2018 Camp Fire in California.

References

- NASA EarthData. "Remote Sensing." 10 December 2024.
<https://www.earthdata.nasa.gov/learn/backgrounders/remote-sensing>.
- UCLA Office of Advanced Research Computing. "Introduction to Remote Sensing With Python." *YouTube*, 9 Feb 2022, <https://www.youtube.com/watch?v=gj4UdFsyoM>.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Brunsdon, Chris, et al. "Geographically Weighted Regression: A Method for Exploring Spatial Nonstationarity." *Geographical Analysis*, vol. 28, no. 4, 1996, pp. 281-298. <https://onlinelibrary.wiley.com/doi/epdf/10.1111/j.1538-4632.1996.tb00936.x>.

- **Required pages:** All 18 pages
- **Estimated reading time:** 2 hours

Recommended Readings

1. Charlton, Martin, and A. Stewart Fotheringham. *Geographically Weighted Regression: A White Paper*. Science Foundation Ireland, 3 March 2009, https://www.geos.ed.ac.uk/~gisteac/fspat/gwr/gwr_arcgis/GWR_WhitePaper.pdf.

Lesson 3 - Geographically Weighted Regression (GWR)

December 25, 2025

MODULE 6 | LESSON 3

1 Geographically Weighted Regression (GWR)

Reading Time	2h
Prior Knowledge	basic statistics, linear regression, Python
Keywords	Spatial analysis, regression, GWR, spatial heterogeneity, kernel

In this lesson, we will be introducing geographically weighted regression as an easy and intuitive way of modeling data that exhibit spatial heterogeneity. We explore the theoretical foundations of GWR, starting with its motivation and how it extends traditional linear regression to account for spatial heterogeneity. In the second half of the notebook, we follow a more hands-on approach with illustrations and a real-world data example in order to complement the theory with the necessary skills one would need in order to use GWR in practice.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

2 1. Geographically Weighted Regression

Geographically weighted regression is an extension of the traditional linear regression that accounts for **spatial heterogeneity** in relationships between variables. This type of regression allows coefficients to vary geographically, capturing the differences in relationship between different points in space. In short, it is a regression that takes into account the location of the dependent variable on the map and assumes that its relation with the independent variables can very well be different from those of a different point on the map.

In order to achieve the above results, the GWR framework performs as many linear regressions as the number of data points on a map. Each of those regressions is a weighted linear regression (WLS) where the weights play the role of a decaying factor: the further one point is from where the WLS takes place, the less important that point becomes.

The first thing that becomes clear is that the coefficients of the GWR will not be constant across the map. On the contrary, for each point, we will end up with a unique set of coefficients, along with an intercept and an error term. But this is exactly the strength of this framework: that it allows for a thorough understanding of how relationships between variables change across space.

Let's start by formulating a traditional linear regression. In the context of GWR, we can refer to the traditional linear regression as “*global*” since the latter approach assumes global (constant) coefficients. The LR formula is:

Traditional Linear Regression

$$y_i = \beta_0 + \sum_{k=1}^p \beta_i \cdot x_{ik} + \epsilon_i$$

where

- y is the dependent variable
- β_i are the coefficients (assumed constant across all observations)
- x_{ik} are the k independent variables and
- ϵ_i are the errors

Geographically Weighted Regression

In GWR, we allow the coefficients to vary by location; thus, they become a function of space: $\beta(u, v)$, where (u, v) are the coordinates.

$$y_i = \beta_0(u_i, v_i) + \sum_{k=1}^p \beta_k(u_i, v_i) x_{ik} + \epsilon_i$$

The above notation hides an important inconvenience: for each point on the map, we only have a single (or maybe a few) observations. This means that it is not possible to reliably estimate the coefficients at each point using the OLS estimator. In order to account for this, we make use of a spatial weighting scheme that encodes the influence of the points to the point of interest. Naturally, the faraway points are assigned a smaller weight than the ones closer. The end result is a diagonal weight matrix W that contains values that range from 0 to 1.

What we actually changed by using a weight matrix here is that for each point on the map, we will estimate β_i using all the available observations weighted in terms of proximity to the point of interest. The estimation of the coefficients can now be given by:

$$\hat{\beta}(u_i, v_i) = [X^T W(u_i, v_i) X]^{-1} X^T W(u_i, v_i) y$$

It follows that the predicted value of each observation is given by:

$$\hat{y}_i = X_i [X^T W(u_i, v_i) X]^{-1} X^T W(u_i, v_i) y$$

If we set:

$$S_i = X_i[X^T W(u_i, v_i)X]^{-1}X^T W(u_i, v_i) \text{ and } C_i = [X^T W(u_i, v_i)X]^{-1}X^T W(u_i, v_i)$$

the local covariance matrix V_i of the parameter estimates becomes:

$$V_i = C_i C_i^T \hat{\sigma}^2$$

where $\hat{\sigma}^2$ is the estimated standard deviation of the error term, which is defined as:

$$\hat{\sigma}^2 = \frac{\sum (y_i - \hat{y}_i)^2}{m - \text{tr}(S)}$$

Let's now check the weighting scheme.

2.0.1 1.1 Weighting Scheme (Data-Borrowing Scheme)

The weighting scheme in GWR is also called a “data-borrowing scheme.” This terminology emphasizes an important aspect of GWR: that spatial proximity-borrowing nearby observations-is essential in estimating local regression coefficients.

The weights assigned to each observation are given by spatial kernel functions. The three most common kernels include the:

- **Gaussian Kernel:**

$$w_{ij} = \exp\left(-\frac{d_{ij}^2}{2b^2}\right)$$

- **Bisquare Kernel:**

$$w_{ij} = \begin{cases} \left(1 - \left(\frac{d_{ij}}{b}\right)^2\right)^2 & \text{if } d_{ij} < b \\ 0 & \text{otherwise} \end{cases}$$

- **Exponential Kernel:**

$$w_{ij} = \exp\left(-\frac{d_{ij}}{b}\right)$$

where

- w_{ij} are the weight for observation j when estimating the model at observation i
- d_{ij} is the distance between i and j
- b is the bandwidth parameter controlling the decay of weights with distance

Distance

The choice of the distance metric between two points is case specific. In many occasions, the Euclidean distance, which measures the straight line between two points, will suffice:

$$d_{ij} = \sqrt{(u_i - u_j)^2 + (v_i - v_j)^2}$$

But if the problem requires that grid-like street patterns are taken into account, then one can use the Manhattan distance:

$$d_{ij} = |u_i - u_j| + |v_i - v_j|$$

If we further assume that the area of interest is large (the curvature of Earth becomes a factor) and/or the landscape includes varied terrains, then geodesic lines or geodesic distance could be more appropriate.

Bandwidth Parameter

The Bandwidth b controls the trade-off between bias and variance by controlling the proximity of the points that will be used in the local regressions. A small bandwidth will emphasize local variations, keeping a small number of observations to estimate the parameters. A large bandwidth can reduce the coefficient variance but might as well miss local characteristics due to the “oversmooth” that will be applied by incorporating far-away points in the analysis. The bandwidth is normally selected with cross validation (CV) or Akaike information criterion (AIC).

Let’s illustrate what the kernels look like in practice. In the next example, we will use a square “map,” which consists of $100 \times 100 = 10,000$ observations. The illustrations include all three kernels and show the differences between using Manhattan and Euclidean distance. For simplicity, the point of interest will be fixed and placed at the center of the map.

```
[2]: # Figure 1
# Create a meshgrid
i_indices = np.arange(100)
j_indices = np.arange(100)
I, J = np.meshgrid(i_indices, j_indices, indexing='ij')

# Manhattan and Euclidean distances
manhattan_distance = np.abs(I - 50) + np.abs(J - 50)
euclidean_distance = np.sqrt((I - 50)**2 + (J - 50)**2)

# Kernel functions
def gaussian_kernel(D, b):
    return np.exp(-(D ** 2) / (2 * b ** 2))

def bisquare_kernel(D, b):
    return ((1 - (D / b) ** 2) ** 2) * (D < b) # Comment this line yourself

def exponential_kernel(D, b):
    return np.exp(- D / b)
```

```

# Plotting function
def plot_kernels(D, bandwidth):
    # Compute weights
    W_gaussian = gaussian_kernel(D, bandwidth)
    W_bisquare = bisquare_kernel(D, bandwidth)
    W_exponential = exponential_kernel(D, bandwidth)

    # Set up the plot
    fig, axs = plt.subplots(1, 3, figsize=(18, 6))

    # Gaussian Kernel
    sns.heatmap(W_gaussian, cmap='viridis', ax=axs[0])
    axs[0].set_title(f'Gaussian Kernel (b = {bandwidth})')

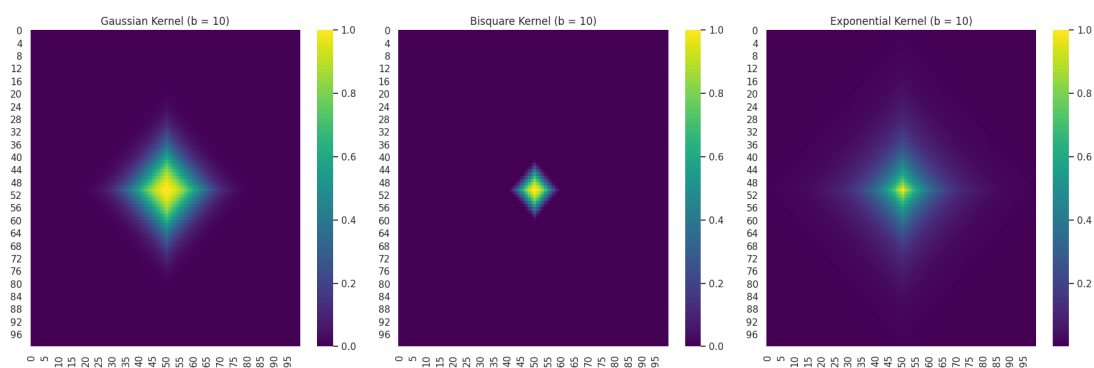
    # Bisquare Kernel
    sns.heatmap(W_bisquare, cmap='viridis', ax=axs[1])
    axs[1].set_title(f'Bisquare Kernel (b = {bandwidth})')

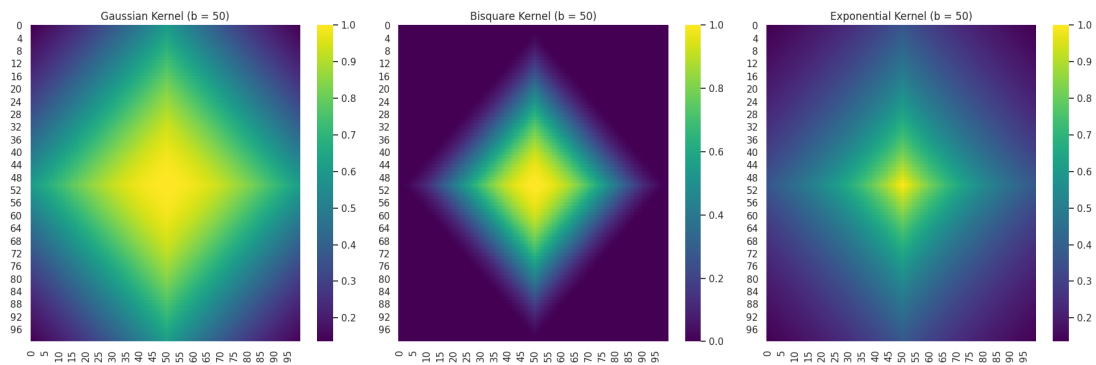
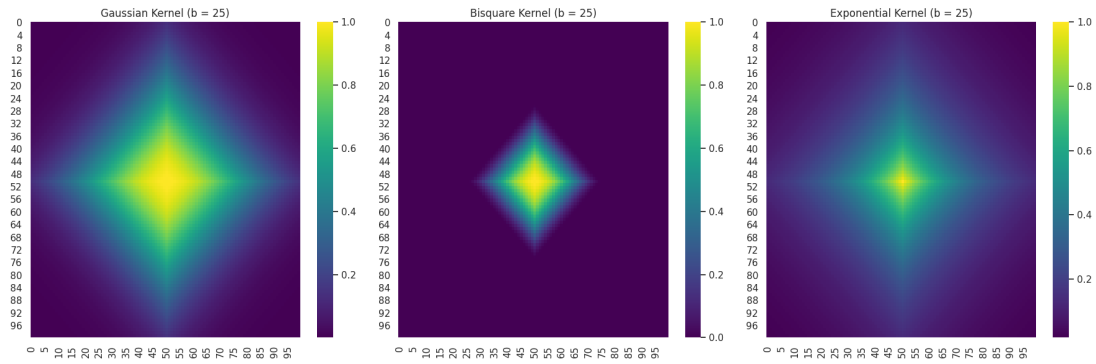
    # Exponential Kernel
    sns.heatmap(W_exponential, cmap='viridis', ax=axs[2])
    axs[2].set_title(f'Exponential Kernel (b = {bandwidth})')

    plt.tight_layout()
    plt.show()

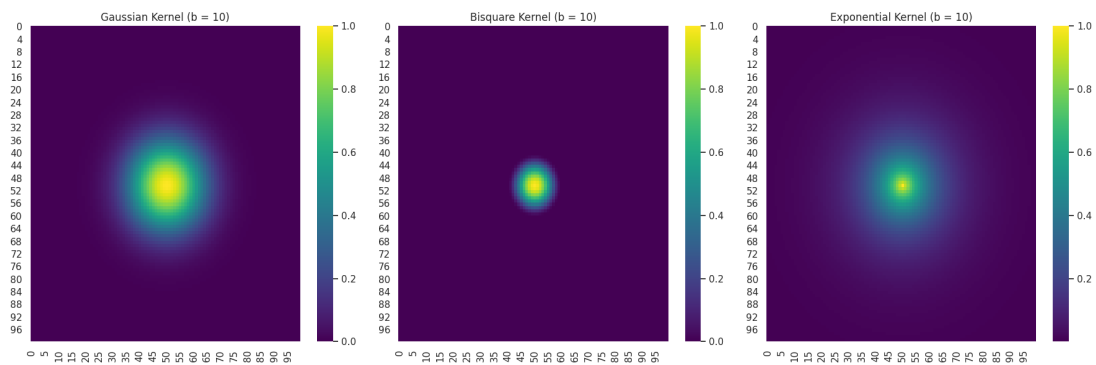
# Plot the kernels using manhattan distance
for b in [10, 25, 50]:
    plot_kernels(D = manhattan_distance, bandwidth=b)

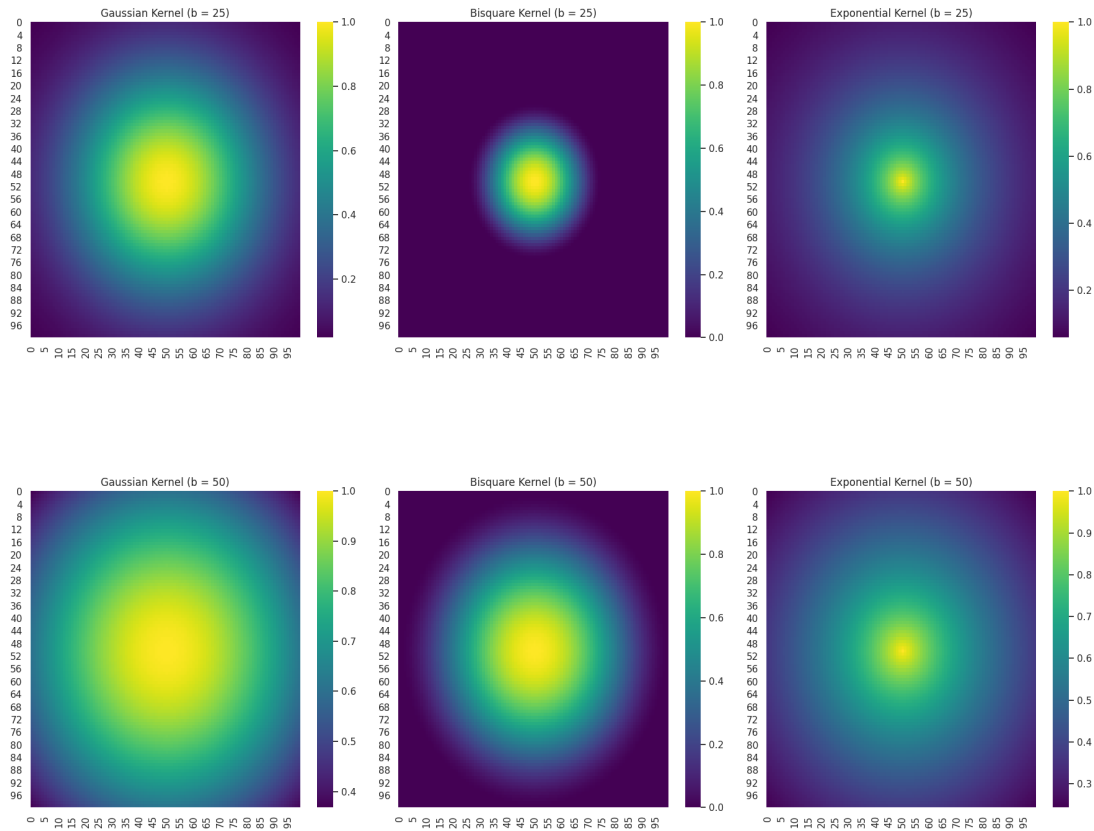
```





```
[3]: # Figure 2
# Plot the kernels using Euclidean distance
for b in [10, 25, 50]:
    plot_kernels(D = euclidean_distance, bandwidth=b)
```





In Figures 1 and 2, we observe the effect of the bandwidth in proximity selection. As the bandwidth increases, the area that is used in the local regression increases as well. We can also see that the shape of the area in each distance metric is different. More importantly, the illustrations make clear the differences between the kernels:

- Gaussian: gradual and smooth decay starting from the point of interest
- Bisquare: sharp cut-off that defines the clear borders of influence
- Exponential: rapid decay but with large tails that account for long-distance effects

3 2. Application of Geographically Weighted Regression (GWR)

In this section, we are going to apply what we learned about GWR from the last section to real-world data. **## 2.1 Importing the Data Source** We will use Airbnb rental data from the Prenzlauer Berg (Prenz) neighborhood in Berlin, Germany, as our example data (Oshan et al.). Prenz is a popular destination for tourists. It provides a lot of dining, art, and night life options for visitors. We will use the data to understand what the key drivers are for the Airbnb rental prices in this neighborhood. This dataset is available through a Python package libpysal. Hence, we will need to install this package first. The other python package we need to install is mgwr. This is the Python package to run GWR. Now let's install these two packages.

```
[4]: pip install libpysal
```

```
Requirement already satisfied: libpysal in /usr/local/lib/python3.11/site-  
packages (4.13.0)  
Requirement already satisfied: beautifulsoup4>=4.10 in  
/usr/local/lib/python3.11/site-packages (from libpysal) (4.12.2)  
Requirement already satisfied: geopandas>=0.10.0 in  
/usr/local/lib/python3.11/site-packages (from libpysal) (1.1.2)  
Requirement already satisfied: numpy>=1.22 in /usr/local/lib/python3.11/site-  
packages (from libpysal) (1.26.2)  
Requirement already satisfied: packaging>=22 in /usr/local/lib/python3.11/site-  
packages (from libpysal) (23.1)  
Requirement already satisfied: pandas>=1.4 in /usr/local/lib/python3.11/site-  
packages (from libpysal) (2.0.0)  
Requirement already satisfied: platformdirs>=2.0.2 in  
/usr/local/lib/python3.11/site-packages (from libpysal) (3.4.0)  
Requirement already satisfied: requests>=2.27 in /usr/local/lib/python3.11/site-  
packages (from libpysal) (2.29.0)  
Requirement already satisfied: scipy>=1.8 in /usr/local/lib/python3.11/site-  
packages (from libpysal) (1.11.4)  
Requirement already satisfied: shapely>=2.0.1 in /usr/local/lib/python3.11/site-  
packages (from libpysal) (2.1.2)  
Requirement already satisfied: scikit-learn>=1.1 in  
/usr/local/lib/python3.11/site-packages (from libpysal) (1.2.2)  
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/site-  
packages (from beautifulsoup4>=4.10->libpysal) (2.4.1)  
Requirement already satisfied: pyogrio>=0.7.2 in /usr/local/lib/python3.11/site-  
packages (from geopandas>=0.10.0->libpysal) (0.12.1)  
Requirement already satisfied: pyproj>=3.5.0 in /usr/local/lib/python3.11/site-  
packages (from geopandas>=0.10.0->libpysal) (3.7.2)  
Requirement already satisfied: python-dateutil>=2.8.2 in  
/usr/local/lib/python3.11/site-packages (from pandas>=1.4->libpysal) (2.8.2)  
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/site-  
packages (from pandas>=1.4->libpysal) (2023.3.post1)  
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.11/site-  
packages (from pandas>=1.4->libpysal) (2023.3)  
Requirement already satisfied: charset-normalizer<4,>=2 in  
/usr/local/lib/python3.11/site-packages (from requests>=2.27->libpysal) (3.1.0)  
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-  
packages (from requests>=2.27->libpysal) (3.4)  
Requirement already satisfied: urllib3<1.27,>=1.21.1 in  
/usr/local/lib/python3.11/site-packages (from requests>=2.27->libpysal)  
(1.26.15)  
Requirement already satisfied: certifi>=2017.4.17 in  
/usr/local/lib/python3.11/site-packages (from requests>=2.27->libpysal)  
(2022.12.7)  
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.11/site-  
packages (from scikit-learn>=1.1->libpysal) (1.3.2)
```

Requirement already satisfied: threadpoolctl>=2.0.0 in
/usr/local/lib/python3.11/site-packages (from scikit-learn>=1.1->libpysal)
(3.2.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-
packages (from python-dateutil>=2.8.2->pandas>=1.4->libpysal) (1.16.0)
WARNING: Running pip as the 'root' user can result in broken permissions
and conflicting behaviour with the system package manager. It is recommended to
use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip
available: 22.3.1 -> 25.3
[notice] To update, run:
pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.

[5]: pip install mgwr

Requirement already satisfied: mgwr in /usr/local/lib/python3.11/site-packages
(2.2.1)
Requirement already satisfied: scipy>=0.11 in /usr/local/lib/python3.11/site-
packages (from mgwr) (1.11.4)
Requirement already satisfied: numpy>=1.3 in /usr/local/lib/python3.11/site-
packages (from mgwr) (1.26.2)
Requirement already satisfied: libpysal>=4.0.0 in
/usr/local/lib/python3.11/site-packages (from mgwr) (4.13.0)
Requirement already satisfied: spglib>=1.0.6 in /usr/local/lib/python3.11/site-
packages (from mgwr) (1.1.0)
Requirement already satisfied: spreg in /usr/local/lib/python3.11/site-packages
(from mgwr) (1.8.4)
Requirement already satisfied: beautifulsoup4>=4.10 in
/usr/local/lib/python3.11/site-packages (from libpysal>=4.0.0->mgwr) (4.12.2)
Requirement already satisfied: geopandas>=0.10.0 in
/usr/local/lib/python3.11/site-packages (from libpysal>=4.0.0->mgwr) (1.1.2)
Requirement already satisfied: packaging>=22 in /usr/local/lib/python3.11/site-
packages (from libpysal>=4.0.0->mgwr) (23.1)
Requirement already satisfied: pandas>=1.4 in /usr/local/lib/python3.11/site-
packages (from libpysal>=4.0.0->mgwr) (2.0.0)
Requirement already satisfied: platformdirs>=2.0.2 in
/usr/local/lib/python3.11/site-packages (from libpysal>=4.0.0->mgwr) (3.4.0)
Requirement already satisfied: requests>=2.27 in /usr/local/lib/python3.11/site-
packages (from libpysal>=4.0.0->mgwr) (2.29.0)
Requirement already satisfied: shapely>=2.0.1 in /usr/local/lib/python3.11/site-
packages (from libpysal>=4.0.0->mgwr) (2.1.2)
Requirement already satisfied: scikit-learn>=1.1 in
/usr/local/lib/python3.11/site-packages (from libpysal>=4.0.0->mgwr) (1.2.2)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/site-

```

packages (from beautifulsoup4>=4.10->libpysal>=4.0.0->mgwr) (2.4.1)
Requirement already satisfied: pyogrio>=0.7.2 in /usr/local/lib/python3.11/site-
packages (from geopandas>=0.10.0->libpysal>=4.0.0->mgwr) (0.12.1)
Requirement already satisfied: pyproj>=3.5.0 in /usr/local/lib/python3.11/site-
packages (from geopandas>=0.10.0->libpysal>=4.0.0->mgwr) (3.7.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.11/site-packages (from
pandas>=1.4->libpysal>=4.0.0->mgwr) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/site-
packages (from pandas>=1.4->libpysal>=4.0.0->mgwr) (2023.3.post1)
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.11/site-
packages (from pandas>=1.4->libpysal>=4.0.0->mgwr) (2023.3)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/site-packages (from
requests>=2.27->libpysal>=4.0.0->mgwr) (3.1.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-
packages (from requests>=2.27->libpysal>=4.0.0->mgwr) (3.4)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in
/usr/local/lib/python3.11/site-packages (from
requests>=2.27->libpysal>=4.0.0->mgwr) (1.26.15)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.11/site-packages (from
requests>=2.27->libpysal>=4.0.0->mgwr) (2022.12.7)
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.11/site-
packages (from scikit-learn>=1.1->libpysal>=4.0.0->mgwr) (1.3.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in
/usr/local/lib/python3.11/site-packages (from scikit-
learn>=1.1->libpysal>=4.0.0->mgwr) (3.2.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-
packages (from python-dateutil>=2.8.2->pandas>=1.4->libpysal>=4.0.0->mgwr)
(1.16.0)

```

WARNING: Running pip as the 'root' user can result in broken permissions
and conflicting behaviour with the system package manager. It is recommended to
use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip
available: 22.3.1 -> 25.3

[notice] To update, run:

`pip install --upgrade pip`

Note: you may need to restart the kernel to use updated packages.

Now, let's import the Python packages we need for this application.

[6]: `pip install folium`

```

Requirement already satisfied: folium in /usr/local/lib/python3.11/site-packages
(0.20.0)

```

Requirement already satisfied: branca>=0.6.0 in /usr/local/lib/python3.11/site-packages (from folium) (0.8.2)
 Requirement already satisfied: jinja2>=2.9 in /usr/local/lib/python3.11/site-packages (from folium) (3.1.2)
 Requirement already satisfied: numpy in /usr/local/lib/python3.11/site-packages (from folium) (1.26.2)
 Requirement already satisfied: requests in /usr/local/lib/python3.11/site-packages (from folium) (2.29.0)
 Requirement already satisfied: xyzservices in /usr/local/lib/python3.11/site-packages (from folium) (2025.11.0)
 Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/site-packages (from jinja2>=2.9->folium) (2.1.2)
 Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/site-packages (from requests->folium) (3.1.0)
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-packages (from requests->folium) (3.4)
 Requirement already satisfied: urllib3<1.27,>=1.21.1 in /usr/local/lib/python3.11/site-packages (from requests->folium) (1.26.15)
 Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/site-packages (from requests->folium) (2022.12.7)
 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip available: 22.3.1 -> 25.3
 [notice] To update, run:
 pip install --upgrade pip
 Note: you may need to restart the kernel to use updated packages.

```
[7]: import pandas as pd
import libpysal as ps
from libpysal.examples import available
from libpysal.examples import load_example
from libpysal.examples import get_path
from mgwr.gwr import GWR, MGWR
from mgwr.sel_bw import Sel_BW
from mgwr.utils import compare_surfaces, truncate_colormap
import geopandas as gp
import folium
```

Pysal contains a lot of example datasets for spatial analysis. For more information about the details of example datasets in this package, you can visit this site [Pysal Example Data](#). We can use the following function to take a look at the list of example datasets from Pysal.

```
[8]: # Check available datasets on Pysal
available()
```

```
[8]:
```

	Name	Description	Installed
0	10740	Albuquerque, New Mexico, Census 2000 Tract Dat...	True
1	AirBnB	Airbnb rentals, socioeconomics, and crime in C...	False
2	Atlanta	Atlanta, GA region homicide counts and rates	False
3	Baltimore	Baltimore house sales prices and hedonics	False
4	Bostonhsg	Boston housing and neighborhood data	False
..	...	...	...
94	taz	Traffic Analysis Zones in So. California	False
95	tokyo	Tokyo Mortality data	True
96	us_income	Per-capita income for the lower 48 US states 1...	True
97	virginia	Virginia counties shapefile	True
98	wmat	Datasets used for spatial weights testing	True

[99 rows x 3 columns]

Now, let's download the Prenz dataset. The title of this dataset on Pysal is called "Berlin," so you can see from the following code that we are downloading the Berlin dataset.

```
[9]: #Download Prenz (Berlin) dataset from Pysal
berlin = load_example('berlin')

# Check if the dataset is intalled and ready for analysis
berlin.installed
```

```
[9]: True
```

3.1 2.2 Visualizing the Data

Okay, we have installed the dataset. Let's take a look at what's inside.

```
[10]: # Check file lists from the installed dataset
berlin.get_file_list()
```

```
[10]: ['/usr/local/lib/python3.11/site-
packages/libpysal/examples/berlin/prenzlauer.zip',
'/usr/local/lib/python3.11/site-packages/libpysal/examples/berlin/README.md',
'/usr/local/lib/python3.11/site-
packages/libpysal/examples/berlin/prenz_bound.zip']
```

From the file lists, we see one file is "prenzlauer.zip" file. This file contains the dataset we need for analysis. The other file is "prenz_bound.zip" file. This file will be used to draw the boundaries for Prenz neighborhood. We are going to use the "get_path" function from the Pysal package and the "read.file" function from the geopandas package to convert these two files into geo-dataframes.

```
[11]: # Convert two zipped files to geo dataframes
prenz = gp.read_file(berlin.get_path('prenzlauer.zip'))
```

```
prenz_bound = gp.read_file(berlin.get_path('prenz_bound.zip'))
```

Let's double-check the types of the two converted files.

```
[12]: # Check the type of Prenz dataset
type(prenz)
```

```
[12]: geopandas.geodataframe.GeoDataFrame
```

```
[13]: # Check the type of Prenz_bound dataset
type(prenz_bound)
```

```
[13]: geopandas.geodataframe.GeoDataFrame
```

Great! We can confirm that both datasets are geo-datasets. Now let's see what both datasets look like.

```
[14]: prenz.head()
```

```
[14]:
```

	accommodat	review_sco	bedrooms	bathrooms	beds	price	X
0	2	100.0	1.0	1.0	1.0	35.0	1.494450e+06
1	2	90.0	1.0	1.0	1.0	23.0	1.494354e+06
2	2	93.0	1.0	1.0	1.0	38.0	1.494406e+06
3	2	100.0	1.0	1.0	1.0	50.0	1.494271e+06
4	2	100.0	1.0	1.0	1.0	80.0	1.493982e+06

	Y	geometry
0	6.899036e+06	POINT (1494450.105 6899036.141)
1	6.899121e+06	POINT (1494353.555 6899120.594)
2	6.898809e+06	POINT (1494405.897 6898808.518)
3	6.898655e+06	POINT (1494270.517 6898655.223)
4	6.899397e+06	POINT (1493982.078 6899397.385)

```
[15]: # Find out number of rental properties in the dataset
len(prenz)
```

```
[15]: 2203
```

The above code shows the first five rows of the data from the Prenz dataset. It also tells us that there are data for 2,203 rental properties in the dataset. Here is a brief description of each variable:

- **accommodat**: Number of visitors the rental can accommodate
- **review_sco**: Cumulative review score from the last visitor who stayed at the rental
- **bedrooms**: Number of bedrooms
- **bathrooms**: Number of bathrooms
- **beds**: Number of beds available
- **price**: Price of the rental
- **X**: Latitude of the rental location
- **Y**: Longitude of the rental location
- **geometry**: Geographic location of the rental

We learned from the previous lesson that the key feature of a geo-dataset is to contain the geographical variable “geometry” to provide the location information for a feature. Because the geographical locations of rentals are point information, we only get one coordinate pair showing latitude and longitude. Now, let’s also take a look at the `prenz_bound` geo dataset.

```
[16]: prenz_bound.head()
```

```
[16]:      neighbourh      geometry
0  Helmholtzplatz  POLYGON ((1496989.669 6899662.266, 1497008.148...
```

As explained before, the `prenz_bound` dataset provides geolocation information to draw the boundaries of the Prenz neighborhood. In this dataset, we can see it provides geolocation information for several points on the map. By connecting all these geolocation points, we can form a region (polygon). This is why we only have a row of data in the `prenz_bound` dataset. Before running any analysis, let’s first draw the rental locations of our dataset on a map.

```
[17]: # Draw Rental Properties of Prenz on a map

# First, create a basemap
map = folium.Map(location=[52.542986,13.427986], zoom_start=13.5,
    ↪control_scale=True)

# Then add the Prenz neighborhood borders to the map
folium.GeoJson(prenz_bound).add_to(map)

# Then add the Airbnb rental locations to the map. We will use a red dot to
    ↪represent the location of one airbnb rental.
folium.GeoJson(prenz,
    ↪marker=folium.Circle(radius=10, fill_color="red", fill_opacity=0.4,
    ↪color="red", weight=1)).add_to(map)

map
```

```
[17]: <folium.folium.Map at 0x70e82940cd50>
```

3.2 2.3 Running the GWR Model

In this section, we are going to use the Prenz data to build a GWR pricing model. We will use Python package `mgwr` for demonstration. For our dependent variable, we will use the logarithm of the price to correctly adjust for skewness of the price data. For independent variables, we will use review scores, number of visitors, and number of bathrooms as three independent variables. The linear equation will look like this. **

$$\log(\text{price}) = \text{intercept} + \text{review scores} + \text{number of visitors} + \text{number of bathrooms}$$

** We will also create a list variable, which includes the u and v variables’ information described in section 1.

```
[18]: #Create variables for Berlin GWR pricing model
#Take the logarithm of the price variable to correct for skewing
b_y = np.log(prenz['price'].values.reshape((-1, 1)))
b_X = prenz[['review_sco', 'accommodat', 'bathrooms']].values
u = prenz['X']
v = prenz['Y']
b_coords = list(zip(u, v))
```

Once we create all the variables we need for the model, our next step is to decide the bandwidth parameter. Python package mgwr provides a Sel_BW function to find the optimal bandwidth. We will use Sel_BW's default optimizing routine and model fit criterion to find the bandwidth parameter. The default kernel and distance selections are bisquare kernel and Euclidean distance. The reason that bisquare kernel is selected is because this kernel will give 0 weights to far-away locations whereas Gaussian and exponential will always give non-zero weights to far-away locations. Then, Sel_BW will use a default search method based on corrected AIC (AICc) to find the optimal bandwidth parameter. **Corrected AIC (AICc)** is a special AIC that is designed to be used with spatial analysis. AICc includes a function to penalize small bandwidth selection due to more complicated models. Okay, now let's find the bandwidth for our model.

```
[19]: #Find bandwidth parameter
selector = Sel_BW(b_coords, b_y, b_X)
bw = selector.search()
print(bw)
```

192.0

Once we obtain the bandwidth, we can use it as an input to build our GWR model.

```
[20]: # Build a GWR model and print out model summary
gwr_model = GWR(b_coords, b_y, b_X, bw)
gwr_results = gwr_model.fit()
gwr_results.summary()
```

```
=====
Model type                                Gaussian
Number of observations:                    2203
Number of covariates:                      4

Global Regression Results
-----
Residual sum of squares:                    406.306
Log-likelihood:                           -1263.871
AIC:                                        2535.742
AICc:                                      2537.769
BIC:                                       -16520.662
R2:                                        0.272
Adj. R2:                                   0.271

Variable                                Est.          SE  t(Est/SE)    p-value
```

X0	3.003	0.112	26.767	0.000
X1	0.004	0.001	3.294	0.001
X2	0.176	0.007	25.472	0.000
X3	0.163	0.037	4.382	0.000

Geographically Weighted Regression (GWR) Results

```

Spatial kernel: Adaptive bisquare
Bandwidth used: 192.000

```

Diagnostic information

```

Residual sum of squares: 320.948
Effective number of parameters (trace(S)): 122.107
Degree of freedom (n - trace(S)): 2080.893
Sigma estimate: 0.393
Log-likelihood: -1004.105
AIC: 2254.424
AICc: 2269.122
BIC: 2955.833
R2: 0.425
Adjusted R2: 0.391
Adj. alpha (95%): 0.002
Adj. critical t value (95%): 3.153

```

Summary Statistics For GWR Parameter Estimates

Variable	Mean	STD	Min	Median	Max
X0	2.853	0.661	1.154	2.830	5.115
X1	0.005	0.006	-0.011	0.005	0.021
X2	0.201	0.068	-0.039	0.217	0.309
X3	0.148	0.214	-0.579	0.167	0.680

The global regression result from the above summary is the OLS model result for our Prenz data. It is used as the benchmark model to compare to our GWR model. The OLS model results include the usual model summary we are familiar with. **## 2.4 Examining the Model Results** The next block of the summary is the GWR model result. First, it tells us the kernel (bisquare) and the bandwidth (192) selected for this model. It also provides a lot of model diagnostic information. For example, the adjusted R2 for OLS model is 0.271 while the adjusted R2 for GWR model is 0.391. GWR model provides a better model fit than OLS model.

Next is the parameter estimation result. In the GWR model, each location will have a set of parameter estimates for independent variables. In our case, we will have 2,203 parameter estimates for each independent variable. In the above summary, X0 is the intercept, X1 is review scores, X2 is the number of accommodated visitors, and X3 is the number of bathrooms. Each of these

variables has 2,203 parameter estimates. Hence, the summary shows the mean, standard deviation, min, median, and max of the parameter estimates for each independent variable. If we want to investigate the details of the estimated parameters, we can use the following Python code. It will show you the set of estimated parameters for each location. Let's look at the estimated parameters for the first five locations.

```
[21]: # Show estimated parameters for
      gwr_results.params[0:5]
```

```
[21]: array([[ 3.30479996e+00,  1.61014298e-03,  2.06243405e-01,
             -2.67321919e-02],
             [ 3.43635569e+00, -4.51930353e-04,  1.95844944e-01,
              5.37474550e-02],
             [ 3.27912291e+00,  2.92768647e-03,  2.12363255e-01,
             -8.10919052e-02],
             [ 2.88779932e+00,  5.06056110e-03,  2.45744327e-01,
              4.65164479e-02],
             [ 2.89309002e+00,  3.06784769e-03,  1.67282256e-01,
              3.42487452e-01]])
```

We know that GWR builds a regression for each location. Hence, each location also has its own R2. Let's take a look at the R2s for the first 10 locations.

```
[22]: gwr_results.localR2[0:10]
```

```
[22]: array([[0.39583113],
             [0.41660429],
             [0.35650484],
             [0.43641022],
             [0.44784271],
             [0.41771337],
             [0.42162229],
             [0.41387561],
             [0.36649332],
             [0.46269692]])
```

We can also create a graph to show the R2s for all locations.

```
[23]: #Show local model fit (R squared) for all locations
      prenz['R2'] = gwr_results.localR2
      prenz.plot('R2', legend = True)
      ax = plt.gca()
      ax.get_xaxis().set_visible(False)
      ax.get_yaxis().set_visible(False)
      plt.title("Local Model Fit")
      plt.show()
```



From the above “Local Model Fit” graph, we can see that the two light regions on the left have the highest concentration of locations with R2s over 0.5. However, in the area left of center in the neighborhood, there are many locations with R2s that are less than 0.2.

3.3 2.5. Demonstration of Calculating GWR Parameters Using Linear Algebra

In this section, we are going to show how to use linear algebra to obtain parameter estimates for GWR. First, let’s review the GWR estimator for local parameter estimates for location i .

$$\hat{\beta}(i) = [X'W(i)X]^{-1} X'W(i)y$$

Let’s use the above formula to calculate the parameter estimates for location 1 (i will be 0 since Python starts at 0). Since the weight matrix is derived through the optimization search method, we will just pull it out from the model results and use it as input for calculation.

```
[24]: #Pull the information of weight matrix of location 1 from GWR model result
W_1 = gwr_results.W[0]
W_matrix = np.diag(W_1)
```

Next, let’s create X matrix. It is the matrix with independent variables as columns. There also needs to be a column of all 1s for intercept estimation. Let’s create this matrix.

```
[25]: # Create X matrix with additional column of all 1s.
b_X_1 = np.hstack([np.ones((b_X.shape[0], 1)), b_X])
```

Now, let's calculate the inverse matrix $[X'W(i)X]^{-1}$ in the equation first.

```
[26]: inverse_matrix = np.linalg.inv(b_X_1.T@W_matrix@b_X_1)
```

Next, let's calculate the parameter estimates for location 1.

```
[27]: beta_1 = inverse_matrix@b_X_1.T@W_matrix@b_y
      beta_1
```

```
[27]: array([[ 3.30479996e+00],
             [ 1.61014298e-03],
             [ 2.06243405e-01],
             [-2.67321919e-02]])
```

Let's compare it with the GWR model results.

```
[28]: gwr_results.params[0]
```

```
[28]: array([ 3.30479996e+00,  1.61014298e-03,  2.06243405e-01, -2.67321919e-02])
```

It turns out both results are the same. This exercise gives us a good idea of how linear algebra is applied to the calculation of a GWR model.

4 3. Conclusion

In this lesson, we studied the theory of geographically weighted regression (GWR). We covered its definition and key concepts. Specifically, we discussed different kernel methods and distance calculation methods for GWR. We also talked about the bandwidth calculation for the model. Then, we used a real-world dataset to demonstrate how to build a GWR model. We also compared GWR model results with traditional OLS model results. Finally, we showed how we can use linear algebra to derive parameter estimates for a GWR model.

4.0.1 References

- Oshan, Taylor M., et al. "MGWR: A Python Implementation of Multiscale Geographically Weighted Regression for Investigating Process Spatial Heterogeneity and Scale." *International Journal of Geo-Information*, vol. 8, no. 6, 2019. pp. 1-31. <https://www.mdpi.com/2220-9964/8/6/269>.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Zhao, Yilin. "Research on the GameStop Stock Prices Surge during Covid-19 under the Perspective of Individual Investors." *Proceedings of the 2021 3rd International Conference on Economic Management and Cultural Industry*, edited by Valentin Vasilev et al., Atlantis Press International B.V., 15 December 2021. <https://www.atlantispress.com/proceedings/icemci-21/125966071>.

- Required pages: All
- Estimated reading time: 15 minutes

2. Hott, Christian. "Leverage and Risk Taking under Moral Hazard." *Journal of Financial Services Research*, vol. 61, 2022, pp. 167–185, <https://link.springer.com/article/10.1007/s10693-021-00359-8>.

- Required pages: Skim tables, charts, and graphs
- Estimated reading time: 10 minutes

3. Wujek, Joseph H. "Challenger O-Ring Data Analysis." Online Ethics Center for Engineering and Science, 1995, <https://onlineethics.org/cases/numerical-design-problems-ethical-content/challenger-o-ring-data-analysis>.

- Required pages: All
- Estimated reading time: 20 minutes

Lesson 4 - Model Failure, Crises, and The Need for Data Analysis

December 25, 2025

MODULE 6 | LESSON 4

1 Model Failure, Crises, and The Need for Data

Reading Time	30 minutes
Prior Knowledge	Financial Markets course
Keywords	Model paradigm, Backfire

In the final lesson of this module, let's recap some of the key lessons we've learned about data and its role in helping to address the financial challenges we discussed in Financial Markets. Specifically, this lesson addresses model failure and crises, motivating the need for data.

Sometimes, models fail. Markets are not the problem; the simplifying representational lens through which we summarize, simplify, or systemize the world is the problem. Models that have worked for decades and that have been tested, validated, and replicated can fail. Models that are proven through derivation and published in peer-reviewed journals can fail. Models that have earned fame, numerous citations, and Nobel Prizes for their authors can fail. Models whose features are found in software and used worldwide can fail. How can this be?

There are all kinds of models. Models exist for predicting volatility and options or for determining risk premium. There are models to trade based on value, which trade relatively infrequently. There are high-frequency trading models that work in fractions of a second. There are microstructural models that model the intricacies of orders on an exchange. There are global macroeconomic models that tackle GDP, employment, inflation, and monetary policy. Models come in all shapes and sizes.

One thing that all models have in common is that they tend to provide insight into the challenges we've discussed: credit risk and financing, volatility, correlation, leverage, non-linearity, liquidity, and regulation. However, their results and interpretations can be misleading when those models fail.

Let's examine ten takeaways from model failure.

2 1. Finance Is Not Physics

The laws of physics are not the laws in finance. Let's be sure to distinguish what works in rational or theoretical models from what works in the real world. Finance is not physics, meaning

financial models reflect human behavior, which itself is a combination of rationality and irrationality. As such, financial models may not work if they assume everyone is behaving rationally. Sometimes, markets oscillate between conditions of great efficiency and conditions that are characterized by extreme cognitive biases. (See Andrew Lo's Adaptive Market Hypothesis https://web.mit.edu/Alo/www/Papers/JPM2004_Pub.pdf for an intriguing discussion of this oscillation).

The irrational part is much newer to the field of economics than the rational assumptions. Only within the last 20 years have Nobel Prizes been awarded to researchers in behavioral economics (e.g., Kahneman, Thaler). Behavioral finance reminds us that many of the assumptions upon which finance is built are flawed in practice. Therefore, there are not immutable laws of finance like there are in the hard sciences. Some of these ideas were captured in a manifesto that was written after the Great Financial Crisis (Wilmott). The manifesto reminds us of the existence of human biases in decision-making, making any so-called financial laws non-reproducible.

Consider the recent price surge in the stock price of Game Stop. Among many lessons we can learn, people can manipulate the stock market for reasons beyond risk return characteristics. Please see and read the article by Zhao (<https://www.atlantispress.com/proceedings/icemci-21/125966071>).

3 2. Leveraged positions can increase gains or aggravate losses.

Suppose you have a well-validated model that extracts alpha. You may be encouraged to use leverage to extract a greater size of that alpha. For example, you could invest 8x as much in the strategy, thinking you will earn 8x the net revenue. You could use leverage to put less down, borrow funds, and produce greater returns. Indeed, this strategy can achieve greater returns—or greater losses! However, one thing it will surely create is greater risk. The increased use of leverage creates more uncertainty around the expected value. Leveraged positions are indifferent to direction: they can increase your gains or deepen the losses.

For more detailed examples, please read Hott's "Leverage and Risk Taking under Moral Hazard" article (<https://www.google.com/url?q=https%3A%2F%2Flink.springer.com%2Farticle%2F10.1007%2Fs10693-021-00359-8>).

4 3. It's easy to mistake variables as parameters: variables change, but parameters are constant.

How do we know what remains constant and what varies in a model? When you model a system, you choose some things as parameters and others as variables. Parameters are constants we estimate from historical data or perhaps imply from market prices. Variables are dynamic quantities that change with time. It's easy to mistake a variable as a parameter. In financial settings, one can easily call a variable a parameter.

Without strong mathematical and statistical skills, we are likely to assume many things are constant: volatility, interest rates, correlations, even distributions. The stronger our analytical toolkit, the greater the extent by which we can successfully relax assumptions and upgrade our models by having quantities like volatility or interest rates be variables instead of parameters. Ironically, we may want some quantities to be parameters instead of variables. The nontechnical person can only rely on models others have built. The technical person can not only use models but they can also adapt models for their own use by relaxing assumptions, relying on more powerful algorithms, and

using advanced methods of calibration and optimization. Moreover, if they add their own creativity, critical thinking, and subject matter expertise, they will be able to create custom models.

This last point justifies the need for a master's program, not just a single course or a video on "how to be a financial engineer." The material provides the theory and implementation of tools that are then used to illustrate problem solving in financial engineering. If a constant volatility model proves to be inadequate (as it is when attempting to reproduce the volatility smile), then we can upgrade the model to a 2-factor stochastic volatility model that requires mathematical know-how, statistical calibration, and computational implementation. The same is true for dynamic interest rates, dynamic correlations, or any other quantity that we wish to express as time varying. The cost of more sophisticated models is the foundation to understanding them, the extra computational complexity to implement them, and the insight to know when they work and when they don't.

5 4. We may not use models correctly, either failing to understand their instructions or limitations.

Models may work properly, but we may not use them properly! We may underestimate a parameter, or we may overestimate it. We may fail to follow instructions correctly or to understand the assumptions imposed by the model. We may use a model under conditions that it didn't expect. It's not that the model is wrong. It's that we're using it incorrectly.

Clearly, one example would be simple negligence. Suppose we are submitting a trade to an exchange. Here, we are using an order as a model. For a given limit order, we specify the price and quantity. Suppose we want to sell 1 unit at 610,000 yen. Imagine if we mixed up price and quantity! Mistakenly, we submit an order to sell 610,000 at 1 yen. Implausible? This happened in December 2005. The order was quite unusual. The price was nowhere near 1 yen, but 610,000 yen which is \$5,041. Also, the number of shares in the order was more than 40 times that amount held by the company. Nevertheless, the exchange did not reject the order. The costly mistake was nearly a quarter of a billion dollars. This type of error is attributed to operational risk. How does the software get placed in production without the oversight of testing and validation? Now imagine a model with thousands of moving parts and trying to assess whether there are fundamental flaws in it.

A more difficult error is if the assumptions or conditions under which a model was built are not followed. Unfortunately, there are tragic illustrations. In 1986, the Space Shuttle Challenger exploded, killing all seven crew members aboard. Investigations followed. The culprit was a piece of equipment—the O-ring. An explanation of what went wrong was given by one of the most brilliant physicists in the world, Richard Feynman. Appearing before a panel to explain what went wrong, Feynman decided to demonstrate the problem. While he was waiting to speak, he put these O-rings in ice water. When it was Dr. Feynman's turn to speak, he took the O-ring out of the ice water and showed that the O-ring lacked the assumed resilience. These O-rings were tested in Florida at warm temperatures, temperatures well above the freezing point. But on that launch day in January, Florida was unseasonably cold—at a temperature low enough where the O-rings were never tested.

This is an example of applying the results of a model at a temperature where it was scarcely tested. In statistical terms, this is known as extrapolation. The testing didn't consider how effectively the O-rings would work in an environment that was 20 degrees colder (11 degrees Celsius colder) than normal. The data and story are explained in the article by Wujek

(<https://www.google.com/url?q=https%3A%2F%2Fonlineethics.org%2Fcases%2Fnumerical-design-problems-ethical-content%2Fchallenger-o-ring-data-analysis>). Please read it. Model failure can be very expensive, with lives and fortunes at stake.

6 5. Liquidity can make models achieve unexpected results.

Suppose you have a great trade. Now you'd like to scale it up by buying more. Taking too big a position starts to question the security's liquidity: is there market impact? It is possible to turn a profitable trade into an unprofitable one by scaling it too much, as the market impact competes with the strategy's alpha. Every trader has a model that extracts alpha from the market. As they trade more and more, they face slippage, the market impact that increases the price entering the trade and lowers the price while exiting the trade. Is there a corresponding model for its market impact? Whether buying or selling, market impact cause prices to move in a way that eats into the strategy's alpha. Left unchecked, the slippage can outweigh any advantage. We'll discuss the Zillow case following this.

7 6. The existence of a model, or the conviction of its author, doesn't mean it works in practice.

"We've been using this model since forever." "This is the model everyone uses." "This model has been around longer than you've been alive!" "Why would I trust your model when famous people and billions of dollars have used this model?"

You're given a model and you're often told that it works. It gives you a full sense of security. You'll see this a lot in this program. For example, in Derivative Pricing, you'll price an option with a model that has been around for 50 years. You compute prices and sensitivities but initially don't question too many things. Over time, you realize some of the assumptions of the model are flawed. For example, you're told volatility is constant. Then, you realize later that's not what is observed in the option market. What makes people cling to models even if they see there is little evidence for it? We'll discuss different cognitive and behavioral biases in Portfolio Management. For example, overconfidence bias can blind someone to facts because of their education, experience, and position. Markets can be humbling.

Just because models are old, Nobel Prize-winning, universal, easily implemented, frequently cited, massively popular, or intuitive doesn't necessarily make them the best model to use. The dynamics of the market change. Consider models for equities. Equities are impacted by central bank decisions. Additions and deletions in the underlying major indices can affect equities. Option expiration has an effect on the underlying equities market. Even the existence and expansion of cryptocurrencies affect how equity markets trade! Valuation models that traditionally work in equities trading may be temporarily disrupted (if not broken) by the expansion and implosion of the cryptocurrency exchanges. As an engineer, you're using data and modeling capabilities to determine sensitivities, structural breaks, or regime shifts. Ironically, you might use more models sometimes to effectively assess whether the models that you have are appropriate. In some drastic conditions or cases, you may need to have discussions with researchers, portfolio managers, and risk managers to determine if the models lack credibility or validation. Blind reliance on untested models can give a false sense of security.

Almost no one gets a memo saying, "This model no longer works. Please use another one." Models

may be used long after their “expiration date.” Learn to think critically about models. Don’t justify use by the appeal to authority fallacy.

8 7. Models assuming linearity may fail due to non-linearities, especially when there is leverage.

Often, we assume a model has linearities: we increase an input, we get a known increase in the output. Suppose the ratio is 1.5. Then, when we double the output, we get triple the output. Other times, we see there are lots of non-linearities. As we’ve seen, convexity causes small changes to produce small results, but bigger changes produce disproportionately bigger results.

Likewise, the piecewise nonlinearity of an option’s payoff can drive an option to be in the money or out of the money. It becomes very important whether the increase brings you on one side of strike or the other! Market impact is an example of a sensitivity that may start out linear (for relatively small amounts traded) and become very non-linear for larger amounts traded. The mistake is usually a gross underestimation of a cost, loss, or risk that can cause a crisis.

9 8. Models are built on assumptions, so be aware of the conditions.

Every model has assumptions. Some people are unaware of these assumptions because they require a deeper level of understanding than simply using the model by running code. There are conditions that may warrant the use (or void the use!) such as Gaussian vs. non-Gaussian, stationary vs. non-stationary, momentum vs. mean-reversion, non-negativity vs negativity. Very often, we have assumptions that can make or break the model. Sometimes, we have conditions that invalidate the model. A common error is running regressions on non-stationary processes and finding what appears to be a meaningful test-statistic, only to realize that this could be the result of spurious regressions. This type of mistake is more of an operator error than a model failure, in that the model was never appropriate to use in the first place. This point makes it essential to build a solid foundation. There are many students and workers who simply go through the motions of running a model without assessing whether it was appropriate in the first place. This failure is easily remedied by studying the underlying mathematical properties of the model.

Other conditions simply relate to the natural boundaries of the target variable in question. A model that predicts a negative stock price is questionable—stock prices cannot be negative. Yet, a model that forbids a negative interest rate is equally questionable—as negative interest rates can and do exist. Likewise, oil price futures can go negative. It’s not always easy to determine which quantities should be strictly non-negative.

10 9. Every model is a paradigm that needs validation to be properly assessed.

Models reflect paradigms. Very often, you will see the world through the very model you are using. For example, if you believe the returns of a stock are Gaussian, then you will call a very large loss a 5 sigma move—a very rare event indeed. Yet, what if the distribution is not truly Gaussian? Five sigma events are highly unlikely. Many return distributions have excess kurtosis, making them

non-Gaussian. So then, we should put away our Gaussian glasses and put on a pair of glasses that views the world through a more appropriate distribution.

Models need validation. It's very hard to know if your model is wrong if somehow you view the world as if the assumptions are automatically true. Banks are allowed to use internal models for running stress tests on their credit risks, but these models need extensive validation. When you look at things like central banks or Basel regulation, they're requiring certain criteria to validate the model.

As a student in the program, you will learn foundational models and then add complexity to them. In the Machine Learning track, for example, you review least squares and then learn numerous extensions. There are two insights that guide this process. First, learning simpler models helps to build the foundation for learning more advanced models. Second, if simple models work well, we can use them. Many times, the simple model may work best! However, if these models underfit, then we have other methodologies we can use. Understanding the "why" and "how" behind models will hone your skill set.

11 10. Models can backfire.

Unfortunately, models can backfire. A model intended to reduce risk may increase it. A model intended to increase returns reduces them in practice. Unfortunately, aside from risk, remember that models may have uncertainty: we may not know the distribution or even the outcomes. Let's consider a real-world example.

Recently, Zillow undertook a strategy itself that backfired. Zillow is an analytics provider of real-estate information. At some point, they decided to undertake a strategy where they would flip residential real estate. After all, they had massive amounts of data. Zillow thought that this data would give them an edge in the marketplace because few other companies had access to this data. This data didn't only include list prices and closing prices, but also traffic, searches, and other data collected from their widely popular search portal. For example, local areas or even streets that seemed to have lots of searches became highly prized properties. Zillow believed they had a keen sense of which properties were popular and would be sold for at or over list price. Furthermore, they had a proprietary algorithm for the valuation of a home—their so-called Zestimate number.

Zillow aggressively bought up properties, attempting to flip them. They purchased homes and thought they could flip them. They did a little market manipulation by gently increasing the prices as they bought, and then they hoped to create a trend that would establish all those homes as being worth more. Unfortunately, the strategy became apparent to the market. It created very bad sentiment that they were manipulating house prices. They also underestimated the lack of diversification: their purchases tended to cluster around hot spots. They also underestimated liquidity risk. It took longer than expected to make improvements and flip the house. They also bought homes a bit late into the bubble, paying near the top of the market. As they sold their inventory, they were selling for an average of about \$80,000 less per house. In total, they lost hundreds of millions of dollars. They had to lay off staff, and they eventually exited the investment business.

Given data, access, experience, and valuation models, they failed to incorporate key issues like correlation, liquidity, and reputation. The advantage they believed to have ultimately backfired into a disastrous investment.

Models can backfire because it's not so easy to know our influence on the market.

12 11. Conclusion

Oftentimes, in their capstone projects, students will run a back test, assuming they can buy and sell at the closing price. They'll use a staggering amount of funds and report extraordinary results that show no market impact or bid-ask spread. They'll assume that their back test on paper reflects exactly what will happen. But by participating in the market, they would definitely encounter bid-ask spread and potentially market impact, depending on the size of their trade.

As previously, one thing that all models have in common is that they tend to provide insight into the challenges we've discussed: credit risk and financing, volatility, correlation, leverage, non-linearity, liquidity, and regulation. However, when misused their results and interpretations are irrelevant at best (that is, simply unusable) and downright misleading at worst. Building that false sense of security can lead to widespread systemic risk.

In this final lesson, we saw models fail for three overarching reasons:

1. they may be specified incorrectly,
2. they may be used incorrectly, and
3. they may be used inappropriately.

With these new insights, let us continue. We will accept these financial challenges. We will leverage our experience of empirical analysis by using data in a variety of models that will take us on multi-semester journey through models from econometrics, time series analysis, stochastic processing, machine learning, deep learning, and more. Whatever models we may be using, let's be mindful of how we use (and attempt not to misuse) them.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.